PUCPR

PONTIFÍCIA UNIVERSIDADE CATÓLICA DO PARANÁ

DATA ANALYTICS / PROJETO BASEADO EM PROBLEMAS

RA1

Ingestão e preparação dos dados

Documentação da entrega

Projeto Perfil das faixas populares nos charts da Deezer por gênero

Curso e turma Bacharelado em Sistemas de Informação • 2º semestre de 2026

Professor ou professora Marina de Lara

Equipe Eduardo Fabri • João Correa • Rodrigo Alves • João Pedro Cardoso

Repositório https://github.com/Jcliz/data-analytics

Entrega Curitiba • 21/10/2026

RA1 / 01

Problema e perguntas analíticas

Defina o contexto do projeto e as perguntas que orientarão a seleção e a preparação dos dados.

Contexto e problema

Os charts da Deezer mostram as faixas mais ouvidas no geral e em cada gênero. Artistas, gravadoras e curadores de playlists os usam para decidir o que lançar e promover, mas o chart traz só a ordem das faixas, não o que elas têm em comum. Este projeto descreve o perfil das faixas do topo de cada gênero: duração, idade do lançamento, popularidade do artista e concentração de poucos artistas no ranking. Saber se esse perfil muda entre gêneros ajuda quem produz e divulga música a comparar a própria faixa com o que se ouve no gênero dela.

Objetivo e recorte

Objetivo: montar um conjunto de dados confiável com as faixas dos charts da Deezer e os atributos da faixa, do álbum e do artista, para comparar o perfil das faixas populares entre gêneros nas etapas seguintes.

Período: um único retrato dos charts, na data da coleta registrada na seção 02 (ficha da fonte). O chart é ao vivo; o projeto não acompanha sua evolução.

Unidade de observação: a faixa (objeto `track` da API). Uma faixa pode estar em mais de um chart; o gênero de comparação é o do chart em que ela aparece.

População coberta: as primeiras posições do chart geral e dos charts de cada gênero listado pela API (`/genre` e `/chart/{id_genero}/tracks`), com o detalhe de cada faixa, álbum e artista.

Limites: só faixas dos charts, não o catálogo inteiro; charts vistos a partir do país da coleta, registrado na seção 02; sem número de reproduções, só o `rank`, índice de popularidade calculado pela Deezer; atributos de áudio como `bpm` ficam fora das perguntas, porque a API costuma devolvê-los zerados quando não os calculou.

Perguntas analíticas

ID	Pergunta a investigar	Dados necessários
P1	Como a duração das faixas varia entre os gêneros dos charts da Deezer na data da coleta?	duration (segundos) da faixa; gênero do chart de origem (/genre, /chart/{id_genero}/tracks)
P2	Que parte das faixas de cada chart de gênero foi lançada nos 12 meses anteriores à coleta?	release_date de /track/{id}; gênero do chart de origem; data da coleta
P3	Faixas de artistas com mais fãs têm rank mais alto, no chart geral e dentro de cada gênero, na data da coleta?	rank da faixa; nb_fan de /artist/{id}; gênero do chart de origem
P4	Quantos artistas distintos ocupam as posições de cada chart de gênero e que fatia dessas posições fica com os 5 artistas com mais faixas?	artist.id do item do chart; position; gênero do chart de origem

Viabilidade inicial

Todos os campos das perguntas existem na API pública da Deezer, sem autenticação: duration, rank, position e artist.id em /chart/{id_genero}/tracks, release_date em /track/{id}, nb_fan em /artist/{id} e os gêneros em /genre (fontes 1 e 2; base/api-deezer.md, seções 2 e 3). Coletar cada chart de gênero registra o gênero de origem em cada item, sem depender do gênero do álbum.

Lacunas a confirmar na seção 03: a faixa pode estar no chart geral e no do seu gênero, e a deduplicação precisa preservar o gênero de origem; release_date e nb_fan podem vir ausentes ou zerados, reduzindo a base de P2 e P3; o gênero do chart, da Deezer, pode diferir do gênero do álbum; e um único retrato não mostra tendência.

RA1 / 02

Fontes de dados e ingestão

Documente a origem dos dados e como eles são carregados no ambiente do projeto. Duplique a ficha para cada fonte.

Ficha da fonte

Identificação F1 • Charts de faixas da Deezer (geral e um por gênero), com os detalhes de cada faixa, álbum e artista.

Origem e acesso Deezer • API pública https://api.deezer.com, sem autenticação nem chave (documentação: https://developers.deezer.com/api) • data de acesso: 16/10/2026.

Formato e volume JSON via API REST, gravado como recebido (57,7 MB no retrato de 16/10/2026) • 2.800 itens em 28 charts, com 2.581 faixas, 2.292 álbuns e 1.688 artistas únicos • atributos da faixa (id, title, duration, rank, explicit_lyrics, preview, position, bpm, isrc, release_date), do álbum (genre_id, genres, label, release_date, record_type, nb_tracks) e do artista (name, nb_fan, nb_album); o df_bruto tem 102 colunas.

Cobertura charts do dia da coleta, sem histórico; cada um traz as 100 faixas mais populares no momento da requisição (total = 100 em todos) • país aparente: US (Estados Unidos) • charts: o geral (gênero 0) e os gêneros de /genre.

Condições de uso API pública; os dados pertencem à Deezer e aos detentores de direitos e são usados só para fins acadêmicos, sem redistribuir as prévias de áudio. A coleta respeita o limite de chamadas com pausa entre requisições.

Relação com o problema

Todos os campos das quatro perguntas (seção 01) vieram no retrato de 16/10/2026 e estão no df_bruto, e cada item guarda o chart de gênero de origem:

- **P1** (duração por gênero): duration e o gênero do chart (chart_id_genero, chart_nome_genero).
- **P2** (lançamentos dos 12 meses anteriores à coleta, por gênero): detalhe_faixa_release_date, o gênero do chart e a data da coleta, no metadados.json.
- **P3** (fãs do artista e rank): rank, detalhe_artista_nb_fan e o gênero do chart.
- **P4** (concentração de artistas por chart): artist_id, position e o gênero do chart.

Por ser de um único dia e país aparente (US), a fonte não apoia perguntas sobre evolução no tempo nem comparações entre países.

Procedimento de ingestão

Etapa	Procedimento ou parâmetro	Evidência no repositório
1	Acesso: GET em https://api.deezer.com, pausa de 0,15 s, até 4 tentativas com espera crescente; o objeto error devolvido com HTTP 200 é tratado (código 4, limite de chamadas, ganha nova tentativa; os demais são registrados)	notebook §2, célula "Parâmetros da coleta e acesso à API"

Etapa	Procedimento ou parâmetro	Evidência no repositório
2	Charts: /genre lista os gêneros e /chart/{id}/tracks traz o geral e cada gênero, paginado por index e limit=100, até o fim ou 300 itens; /infos registra o país aparente	notebook §2, célula "Coleta dos charts por gênero"
3	Detalhes: /track/{id}, /album/{id} e /artist/{id} para cada faixa, álbum e artista únicos	notebook §2, célula "Coleta dos detalhes de faixas, álbuns e artistas"
4	Retrato bruto: respostas cruas em dados/brutos/AAAA-MM-DD/ (infos.json, generos.json, charts.json, faixas.json, albuns.json, artistas.json) e metadados.json (início e fim, país, parâmetros, chamadas por endpoint, novas tentativas e erros). As etapas 2 a 4 só rodam com BAIXAR_DA_API = True	notebook §2, célula "Gravação do retrato bruto"; dados/brutos/2026-10-16/
5	Leitura: com BAIXAR_DA_API = False (padrão), lê o retrato mais recente de dados/brutos/ e mostra o metadados.json	notebook §2, células "Leitura do retrato bruto mais recente" e "Metadados do retrato lido"
6	df_bruto: uma linha por item de chart, com a origem (chart_id_genero, chart_nome_genero, chart_ordem) e os campos de faixa, álbum e artista juntados pelo id (prefixos detalhe_faixa_, detalhe_album_, detalhe_artista_), sem limpeza e com as repetições entre charts	notebook §2, célula "Montagem do df_bruto"
7	Verificação: linhas contra os metadados, faixas, álbuns e artistas únicos, itens sem detalhe, colunas por origem e itens por chart	notebook §2, célula "Verificação do carregamento"

Resultado da ingestão

A coleta de 16/10/2026 (21h03 a 21h41 UTC) fez 6.591 chamadas, sem novas tentativas nem erros (dados/brutos/2026-10-16/metadados.json). Os 28 charts (o geral e 27 gêneros) vieram com 100 itens cada, numa página: é o que a API oferece (total = 100), abaixo do teto de 300. O df_bruto tem 2.800 linhas e 102 colunas, com 2.581 faixas, 2.292 álbuns e 1.688 artistas únicos, e nenhum item sem detalhe (notebook §2, célula "Verificação do carregamento"). O retrato fica versionado em dados/brutos/AAAA-MM-DD/ e toda execução lê o mesmo. A única etapa manual é a coleta, feita uma vez no Google Colab ou na máquina de um integrante (a API exige acesso direto à internet), com o retrato enviado ao repositório.

RA1 / 03

Compreensão e qualidade

Descreva o que cada registro representa e apresente evidências das verificações de qualidade relevantes para os dados.

Unidade de observação ocorrência de faixa num chart de origem, no retrato de 16/10/2026: 2.800 ocorrências, 102 atributos, 2.581 faixas, 28 charts (geral e 27 gêneros). Evidência: notebook §3, célula “Visão geral e tipos observados”, sobre dados/brutos/2026-10-16/.

Identificação dos registros id identifica a faixa e se repete entre charts; a chave é o par chart_id_genero e id, com zero excedentes em 2.800 (retratos futuros exigirão a data na chave). A posição pertence ao chart, não à faixa. Evidência: notebook §3, célula “Duplicidade e limite da chave”.

Dicionário inicial dos dados

Atributos usados pelas P1–P4, pelo RA2 e por este diagnóstico; os demais (URLs, capas, países, colaboradores) ficam no bruto. Os 102 atributos e suas ausências estão nas células “Visão geral e tipos observados” e “Ausências nulas e textos vazios”. Tipos do pandas 2.3.3; textos são object, sem conversão de data.

Atributo	Significado	Tipo e formato	Unidade ou domínio
chart_id_genero	Id do chart de origem; grupo das P1–P4	int64	Inteiro ≥ 0; 0 = geral
chart_nome_genero	Nome do chart; grupo das P1–P4 e gênero do evento RA2	object	Texto; All = geral
chart_ordem	Ordem da ingestão; verifica a posição (P4)	int64	Inteiro positivo, por chart
id	Id da faixa nas P1–P4; origem de id_faixa no RA2	int64	Inteiro positivo; repete entre charts
title	Título; identificação nas P1–P4	object	Texto
title_version	Versão; ausência informativa	object	Texto ou nulo; vazio permitido
duration	Duração (P1); limite da duração ouvida no RA2	int64	Segundos, inteiro positivo
rank	Popularidade (P3); peso do sorteio no RA2	int64	Inteiro ≥ 0; não é a posição
position	Posição no chart (P4)	int64	Inteiro positivo
explicit_lyrics	Conteúdo explícito; verificação de tipo	bool	Booleano

Atributo	Significado	Tipo e formato	Unidade ou domínio
preview	URL da prévia; ausência, sem uso nas P1–P4	object	Texto, pode ser vazio
artist_id	Id do artista (P3 e contagem da P4)	int64	Inteiro positivo
artist_name	Nome do artista (P3–P4, evento RA2)	object	Texto; agrupar pelo id
album_id	Id do álbum; chave do enriquecimento	int64	Inteiro positivo
detalhe_faixa_id	Id de /track; integridade da junção	int64	Domínio de id
detalhe_faixa_duration	Duração em /track; consistência com duration	int64	Segundos, inteiro positivo
detalhe_faixa_rank	Rank em /track; consistência com rank	int64	Inteiro ≥ 0
detalhe_faixa_release_date	Lançamento da faixa (P2), não do álbum	object	AAAA-MM-DD; até a coleta
detalhe_faixa_bpm	Andamento; diagnóstico, sem uso nas P1–P4	float64	BPM, real; zero a avaliar
detalhe_faixa_gain	Ganho; diagnóstico, sem uso nas P1–P4	float64	dB, real; negativo válido, zero a avaliar
detalhe_album_id	Id de /album; integridade da junção	int64	Domínio de album_id
detalhe_album_genre_id	Gênero do álbum; não é o grupo das P1–P4	int64	Inteiro; ≤ 0 é genérico
detalhe_album_release_date	Lançamento do álbum; formato, não substitui a data da P2	object	AAAA-MM-DD; até a coleta
detalhe_artista_id	Id de /artist; integridade da junção	int64	Domínio de artist_id
detalhe_artista_nb_fan	Fãs do artista (P3)	int64	Inteiro ≥ 0

Diagnóstico de qualidade

Denominador: **2.800 ocorrências**; as verificações não se somam e repetições contam só os excedentes.

Q1 cobre id, chart_id_genero, chart_nome_genero, title, duration, rank, position, artist_id, artist_name, album_id, detalhe_faixa_release_date e detalhe_artista_nb_fan. Q7 espera inteiros nas chaves e contagens, reais em BPM e gain e booleano em explicit_lyrics. Q8 sinaliza

não finitos, valores não positivos em duração, posição e ids, e negativos em rank, fãs ou BPM; não há teto de rank, e gain negativo é válido.

Em Q11, a faixa 4346251442, “Dirty Bird”, tem data 2026-10-28 (faixa e álbum), posterior à coleta de 16/10; talvez lançamento anunciado, causa não confirmada. Em Q17, os limites são 66,5 a 366,5 s, e as durações vão de 41 a 2.120 s. Em Q18, os limites de rank são −445.721,75 a 1.316.536,25. Evidências: células “Datas, formatos e categorias” e “Possíveis outliers e integridade do enriquecimento”.

Uma ocorrência não tem capa nem lista de faixas do álbum (1/2.800; 0,04%), metadados fora das P1–P4 e do RA2. Em title_version, há 57 nulos e 2.476 vazios: versão não informada, sem motivo para excluir faixa. Evidência: célula “Ausências nulas e textos vazios”.

ID	Verificação e regra	Resultado observado	Evidência
Q1	Ausência em campo essencial às P1–P4 ou RA2	0/2.800 (0,00%)	“Ausências nulas e textos vazios”
Q2	Ausência de preview (nulo ou vazio)	1/2.800 (0,04%)	“Ausências nulas e textos vazios”
Q3	Ausência de title_version (nulo ou vazio)	2.533/2.800 (90,46%)	“Ausências nulas e textos vazios”
Q4	Excedentes por id	219/2.800 (7,82%)	“Duplicidade e limite da chave”
Q5	Excedentes pelo par chart_id_genero e id	0/2.800 (0,00%)	“Duplicidade e limite da chave”
Q6	Linhas inteiras repetidas	0/2.800 (0,00%)	“Duplicidade e limite da chave”
Q7	Tipo divergente do inteiro, real ou booleano esperado	0/2.800 (0,00%)	“Estatísticas descritivas e domínios numéricos”
Q8	Numéricos não finitos, ausentes ou fora do domínio	0/2.800 (0,00%)	“Estatísticas descritivas e domínios numéricos”
Q9	bpm igual a zero (dado não informado)	1.890/2.800 (67,50%)	“Estatísticas descritivas e domínios numéricos”
Q10	gain igual a zero (a avaliar)	27/2.800 (0,96%)	“Estatísticas descritivas e domínios numéricos”
Q11	Data fora de AAAA-MM-DD, inválida ou fora de 1900 até a coleta	1/2.800 (0,04%)	“Datas, formatos e categorias”
Q12	Espaços nas pontas de título, artista ou gênero	1/2.800 (0,04%)	“Datas, formatos e categorias”

ID	Verificação e regra	Resultado observado	Evidência
Q13	Mais de um nome por id de artista	0/2.800 (0,00%)	“Datas, formatos e categorias”
Q14	Gênero com mais de um nome, ou nome com mais de um id	0/2.800 (0,00%)	“Datas, formatos e categorias”
Q15	Chart geral (id 0), que não é gênero	100/2.800 (3,57%)	“Datas, formatos e categorias”
Q16	Gênero do álbum genérico (id ≤ 0)	11/2.800 (0,39%)	“Datas, formatos e categorias”
Q17	Duração fora de Q1 − 1,5 IQR a Q3 + 1,5 IQR	143/2.800 (5,11%)	“Possíveis outliers e integridade do enriquecimento”
Q18	Rank fora de Q1 − 1,5 IQR a Q3 + 1,5 IQR	0/2.800 (0,00%)	“Possíveis outliers e integridade do enriquecimento”
Q19	Id ausente ou divergente na junção faixa/álbum/artista	0/2.800 (0,00%)	“Possíveis outliers e integridade do enriquecimento”
Q20	Duração do chart diferente da de /track	0/2.800 (0,00%)	“Possíveis outliers e integridade do enriquecimento”
Q21	Rank do chart diferente do de /track	0/2.800 (0,00%)	“Possíveis outliers e integridade do enriquecimento”

Impacto dos problemas identificados

Prioridade: preservar os grupos das P1–P4. Q4 não autoriza descartar as 219 ocorrências excedentes; a preparação mantém o vínculo faixa-chart. Q15 exige separar o chart geral dos gêneros, e Q16 mostra que o gênero do álbum não substitui o de origem. Q11 pede decisão para o lançamento futuro não contar nos 12 meses da P2.

Q9 e Q10 são secundários: P1–P4 e o RA2 não usam BPM nem gain. Q12 pede inspecionar antes de padronizar; Q17, revisar os extremos sem excluí-los, pois IQR não prova erro. Q2 não afeta as perguntas.

Q1, Q5–Q8, Q13–Q14 e Q18–Q21 não acharam ocorrências nos testes executados. As decisões estão na seção 04; esta seção não altera `df_bruto`.

RA1 / 04

Limpeza e preparação

Relacione cada intervenção ao diagnóstico. Justifique tanto os tratamentos realizados quanto as decisões de manter os dados.

Registro das decisões

Decisão e diagnóstico D1 — Preservação das participações nos charts; Q4, Q5 e Q6.

Problema e abrangência

219/2.800 ocorrências excedentes por id, sem duplicata chart-faixa nem linha igual: a faixa aparece em vários charts. Evidência: notebook §3, "Duplicidade e limite da chave".

Tratamento e justificativa

Uma linha por par chart-faixa: excluir pelo id apagaria participações das P1–P4. Nenhum dos 2.800 registros saiu; a chave final inclui a data da coleta.

Implementação notebook §4, célula "Preservação do vínculo entre faixa e chart".

Validação após o tratamento

Chave composta com 0/2.800 duplicatas e 219/2.800 excedentes por id mantidos. Evidência: notebook §5, "Comparação antes e depois da preparação".

Decisão e diagnóstico D2 — Padronização dos espaços em textos; Q12.

Problema e abrangência

1/2.800: "Sona Mohapatra" com espaço não separável no fim. Evidência: notebook §3, "Datas, formatos e categorias".

Tratamento e justificativa

Espaços nas pontas removidos de título, artista e chart, preservando acentos e maiúsculas; um campo mudou, nenhum registro saiu.

Implementação notebook §4, célula "Padronização dos campos textuais".

Validação após o tratamento

Texto antes e depois exibido; de 1/2.800 para 0/2.800. Evidência: notebook §5, "Comparação antes e depois da preparação".

Decisão e diagnóstico D3 — Conversão e sinalização das datas de lançamento; Q11.

Problema e abrangência

1/2.800: faixa 4346251442, "Dirty Bird", lançada em 28/10/2026, após a coleta de 16/10/2026; causa não confirmada. Evidência: notebook §3, "Datas, formatos e categorias".

Tratamento e justificativa

Lançamento convertido para data sem mudar o valor; criados data_coleta (metadados, UTC) e lancamento_posterior_coleta, que separa o futuro dos já ocorridos na P2. Sem exclusão nem estimativa.

Implementação notebook §4, célula "Conversão e validação temporal", e §5, "Padronização dos nomes e tipos".

Validação após o tratamento

2.800 datas convertidas sem perda (formato, calendário e texto conferidos), a futura sinalizada e preservada na releitura. Evidência: notebook §5, "Gravação e releitura do conjunto preparado".

Decisão e diagnóstico D4 — Identificação do chart geral e preservação da origem; Q14, Q15 e Q16.

Problema e abrangência

100/2.800 do chart geral e 11/2.800 com gênero genérico no álbum; ids e nomes de chart consistentes. Evidência: notebook §3, "Datas, formatos e categorias".

Tratamento e justificativa

Id e nome do chart mantidos e tipo_chart (geral ou genero) criado; o geral serve à P3 e o gênero do álbum não substitui a origem das P1–P4. Sem exclusão nem gênero estimado.

Implementação notebook §4, célula "Identificação do chart de origem".

Validação após o tratamento

100/2.800 com id_chart zero como geral e 2.700/2.800 como genero, origens preservadas. Evidência: notebook §5, "Verificações finais do conjunto preparado".

Decisão e diagnóstico D5 — Seleção dos atributos do conjunto preparado; Q2, Q3, Q9 e Q10.

Problema e abrangência

Uma prévia e 2.533 versões ausentes, 1.890 BPM e 27 ganhos zerados, em campos fora das P1–P4 e do RA2. Evidências: notebook §3, "Ausências nulas e textos vazios" e "Estatísticas descritivas e domínios numéricos".

Tratamento e justificativa

12 dos 102 atributos mais quatro de contexto ou qualidade: 16 colunas. Os 90 restantes (prévia, versão, BPM, ganho, álbum) ficam no bruto, sem imputação nem exclusão.

Implementação notebook §5, célula "Seleção e organização dos atributos".

Validação após o tratamento

16 colunas do dicionário, sem nulos nos 2.800 registros, com os campos das P1–P4 e os cinco nomes do gerador do RA2. Evidência: notebook §5, "Verificações finais do conjunto preparado".

Decisão e diagnóstico D6 — Padronização dos nomes e tipos; Q7 e Q8.

Problema e abrangência

Sem problemas de tipo ou domínio nas 2.800, mas nomes da API e datas em texto a organizar. Evidência: notebook §3, "Estatísticas descritivas e domínios numéricos".

Tratamento e justificativa

Nomes em português (rank mantido pelo gerador): oito inteiros, quatro textos, duas datas e dois booleanos. Valores válidos intactos; falha de conversão não vira zero.

Implementação notebook §5, célula "Padronização dos nomes e tipos".

Validação após o tratamento

Inteiros conferidos (ausência, finitude, fração) antes da conversão; domínios, valores e tipos mantidos na releitura. Evidências: notebook §5, "Verificações finais do conjunto preparado" e "Gravação e releitura do conjunto preparado".

Decisão e diagnóstico D7 — Preservação e sinalização dos extremos de duração; Q17 e Q18.

Problema e abrangência

143/2.800 durações fora do IQR (66,5 a 366,5 s) e 0/2.800 ranks. Evidência: notebook §3, "Possíveis outliers e integridade do enriquecimento".

Tratamento e justificativa

Valores mantidos e duracao_fora_limites_iqr criado: extremo não comprova erro, e cortá-lo alteraria a distribuição da P1.

Implementação notebook §4, célula "Sinalização dos extremos de duração".

Validação após o tratamento

143/2.800 sinalizadas pela regra de Q17 e Q18 (seção 03), valores iguais à origem. Evidência: notebook §5, "Verificações finais do conjunto preparado".

Decisão e diagnóstico D8 — Manutenção e revalidação dos dados aprovados; Q1, Q13 e Q19–Q21.

Problema e abrangência

0/2.800 problemas em campos essenciais, nomes por artista e junções (inclusive duração e rank). Evidências: notebook §3, "Ausências nulas e textos vazios", "Datas, formatos e categorias" e "Possíveis outliers e integridade do enriquecimento".

Tratamento e justificativa

Valores aprovados mantidos e testes repetidos, sem imputação; o df_bruto ficou intacto (a preparação usou uma cópia).

Implementação notebook §5, célula "Verificações finais do conjunto preparado".

Validação após o tratamento

2.800 registros aprovados em obrigatórios, chave, domínios, nomes, datas, sinalizações e integridade; valores iguais ao bruto e releitura igual à memória. Evidência: notebook §5, "Gravação e releitura do conjunto preparado".

Resumo das intervenções

Decisão	Ação executada	Impacto nos dados	Evidência
D1	Preservar o vínculo chart-faixa	2.800/2.800 mantidos; 219/2.800 excedentes por id preservados	Notebook §4, "Preservação do vínculo entre faixa e chart"
D2	Remover espaços nas pontas	De 1/2.800 para 0/2.800	Notebook §4, "Padronização dos campos textuais"
D3	Converter e sinalizar datas	2.800 convertidas; 1/2.800 futura mantida e sinalizada	Notebook §4, "Conversão e validação temporal"
D4	Identificar o tipo de chart	100/2.800 do geral identificadas e preservadas	Notebook §4, "Identificação do chart de origem"
D5	Selecionar atributos	12 de origem, 90 não selecionados e quatro novos; 16 finais	Notebook §5, "Seleção e organização dos atributos"
D6	Padronizar nomes e tipos	16 colunas: oito inteiros, quatro textos, duas datas, dois booleanos	Notebook §5, "Padronização dos nomes e tipos"
D7	Sinalizar extremos	143/2.800 sinalizadas, sem exclusão ou alteração	Notebook §4, "Sinalização dos extremos de duração"
D8	Revalidar consistência	2.800/2.800 aprovados pelas regras finais	Notebook §5, "Verificações finais do conjunto preparado"

RA1 / 05

Conjunto de dados resultante

Identifique a saída do RA1 e demonstre sua consistência para uso nas próximas etapas.

Arquivo ou conjunto final dados/preparados/faixas_preparadas.csv, CSV UTF-8, separador vírgula, sem coluna de índice: 2.800 registros, 16 atributos e 367.932 bytes. Evidência: notebook §5, célula "Gravação e releitura do conjunto preparado", e o próprio CSV.

Unidade de observação final uma ocorrência de faixa em um chart de origem na data da coleta, com chave data_coleta, id_chart e id_faixa; a mesma faixa em charts diferentes não é duplicata. Evidência: notebook §5, célula "Verificações finais do conjunto preparado".

Dicionário atualizado dados/preparados/dicionario-faixas.md, com origem, significado, tipo e domínio dos 16 atributos: oito inteiros, quatro textos, duas datas e dois booleanos. rank manteve o nome por compatibilidade com o gerador de eventos; os demais foram padronizados em português. Evidência: notebook §5, célula "Padronização dos nomes e tipos".

Comparação antes e depois

Evidência: notebook §5, célula "Comparação antes e depois da preparação", sobre o retrato de 16/10/2026. As medidas usam 2.800 registros como denominador e podem se sobrepor; não devem ser somadas.

Medida	Após ingestão	Após preparação	Explicação da diferença
Registros	2.800	2.800	Ocorrências preservadas, sem exclusão (D1)
Atributos	102	16	12 atributos de origem e quatro campos de contexto ou qualidade (D5)
Ocorrências com espaços nas pontas	1/2.800 (0,04%)	0/2.800 (0,00%)	Correção textual sem alterar a escrita (D2)
Duplicatas pela chave chart-faixa	0/2.800 (0,00%)	0/2.800 (0,00%)	Unicidade mantida em cada chart e retrato (D1)
Ocorrências excedentes por id da faixa	219/2.800 (7,82%)	219/2.800 (7,82%)	Participações em charts distintos mantidas (D1)
Ocorrências do chart geral	100/2.800 (3,57%)	100/2.800 (3,57%), identificadas	Preservadas e distinguidas pelo tipo de chart (D4)
Lançamentos posteriores à coleta	1/2.800 (0,04%)	1/2.800 (0,04%), sinalizado	Valor da fonte preservado, sem imputar data (D3)

Medida	Após ingestão	Após preparação	Explicação da diferença
Durações fora dos limites IQR	143/2.800 (5,11%)	143/2.800 (5,11%), sinalizadas	Valores preservados, com sinalização de qualidade (D7)

Verificações finais

Os 2.800 registros passaram nas verificações abaixo (notebook §5, células "Padronização dos nomes e tipos" e "Verificações finais do conjunto preparado"):

- As 16 colunas seguem o dicionário, sem nulos nos campos selecionados.
- A chave data-chart-faixa tem 0/2.800 duplicatas e preserva os vínculos de origem.
- Ids de faixa, artista e álbum, posição e duração são inteiros positivos; id_chart, rank e número de fãs, inteiros não negativos.
- Textos sem vazios nem espaços nas pontas; um nome de chart e um nome de artista por id.
- Datas convertidas sem perda; a data da coleta vem dos metadados, e as sinalizações seguem as regras temporal e IQR.
- Identificadores, duração, rank e fãs mantêm os valores de origem, e as junções com faixa, álbum e artista seguem consistentes.
- O df_bruto, comparado com a cópia anterior à preparação, permaneceu intacto.

O CSV foi relido com tipos informados e datas convertidas, e o relido é igual ao conjunto em memória em 2.800 linhas e 16 colunas (notebook §5, célula "Gravação e releitura do conjunto preparado"). Como CSV não guarda tipos, futuras leituras devem reaplicá-los conforme o dicionário.

Limitações e pendências

O conjunto é o retrato de 16/10/2026, com país aparente US: não é série histórica nem permite comparar países. Evidência: dados/brutos/2026-10-16/metadados.json.

A causa da data futura (1/2.800) não foi confirmada; ela foi preservada e sinalizada, e na P2 não deve contar como lançamento já ocorrido nos 12 meses anteriores à coleta. As 143/2.800 durações fora dos limites IQR também foram mantidas e sinalizadas, pois o critério estatístico não comprova erro. Evidências: notebook §4, células "Conversão e validação temporal" e "Sinalização dos extremos de duração".

BPM, ganho, prévia, versão complementar e outros metadados ficaram fora do CSV, mas seguem no retrato bruto; a seleção não os corrige nem estima. Evidência: notebook §5, célula "Seleção e organização dos atributos".

As 100/2.800 ocorrências do chart geral seguem disponíveis para a P3. Os eventos por gênero do RA2 usam só tipo_chart == "genero", 2.700/2.800 ocorrências; o filtro é aplicado na seção 9. Evidência: notebook §4, célula "Identificação do chart de origem".

RA1 / 06

Organização e reprodutibilidade

Explique como outra pessoa pode obter os dados e executar o fluxo completo do RA1.

Repositório e versão https://github.com/Jcliz/data-analytics • tag entrega, no commit da execução final (o PDF entregue fica em output/entrega-ra1-ra2.pdf nessa tag)

Ambiente Google Colab (referência) ou Python 3.9 ou mais novo em máquina local • versões mínimas em requirements.txt: pandas 2.2 a 2.x e requests 2.31 ou mais novo (RA1), pyspark 4.0.3 com Java 17 ou mais novo (RA2), jupyter, nbconvert, python-docx e pypdf (notebook e PDFs) • conferidas em 16/10/2026 numa instalação limpa: Python 3.13.16, pandas 2.3.3, requests 2.34.2, pyspark 4.0.3, nbconvert 7.17.2, python-docx 1.2.0 e pypdf 6.21.0

Localização dos artefatos

Artefato	Caminho ou forma de obtenção
Dados brutos	dados/brutos/2026-10-16/, versionado: respostas cruas da API em JSON (infos.json, generos.json, charts.json, faixas.json, albuns.json, artistas.json, 57,7 MB) e metadados.json (horário, país aparente US, 6.591 chamadas, 0 erros). Nova coleta só com BAIXAR_DA_API = True na seção 00b
Código de ingestão e preparação	notebook/projeto_data_analytics.ipynb, gerado por python scripts/montar_notebook.py a partir de notebook/secoes/ (ambiente em 00b, ingestão em 02, exploração em 03, limpeza em 04, transformação em 05)
Dados preparados	dados/preparados/faixas_preparadas.csv (UTF-8, separador vírgula), regravado pelas seções 04 e 05 a cada execução; nunca editado à mão
Documentação e dependências	README.md (seção "Como executar"), este documento em documentacao/ra1/ (gerado por python scripts/gerar_docx_ra1.py), dados/README.md, requirements.txt e docs/adr/ (decisões)

Passos para reprodução

A reprodução não depende da API (fora do ar, bloqueada ou com outro chart conforme o país e o dia): o notebook lê por padrão o retrato versionado de 16/10/2026, base dos números deste documento. Uma coleta nova (BAIXAR_DA_API = True) grava outra pasta em dados/brutos/, lida por ser a mais recente, e muda todas as contagens; para voltar ao retrato da entrega, apague a pasta nova ou use um clone limpo.

Ordem	Instrução executável	Resultado esperado
1	Preparar o ambiente. No Colab: *Arquivo → Abrir notebook → GitHub*, repositório Jcliz/data-analytics, arquivo notebook/projeto_data_analytics.ipynb. Localmente: git clone https://github.com/Jcliz/data-analytics.git, cd data-analytics, python -m venv .venv, ativar e pip install -r requirements.txt	No Colab, o notebook abre com o RA1 e o RA2. Localmente, pip e python -c "import pandas, requests" terminam sem erro
2	Obter os dados, sem acessar a API: a seção 00b clona o repositório no Colab com BAIXAR_DA_API = False, e a seção 02 lê o retrato versionado em dados/brutos/	"Preparação do ambiente" mostra a raiz do projeto; "Leitura do retrato bruto mais recente", dados/brutos/2026-10-16 e os sete JSON
3	Executar do início ao fim. No Colab: *Ambiente de execução → Executar tudo*. Localmente: python scripts/montar_notebook.py e jupyter nbconvert --to notebook --execute --inplace notebook/projeto_data_analytics.ipynb	Nenhuma célula com erro; na seção 2, df_bruto: 2800 linhas × 102 colunas, 2.581 faixas, 2.292 álbuns, 1.688 artistas e nenhum item sem detalhe ("Verificação do carregamento")
4	Validar a saída: conferir o conjunto preparado e rodar python scripts/verificar_entrega.py	dados/preparados/ faixas_preparadas.csv regravado (seção 05); script sem pendência de notebook desatualizado ou não executado

Verificação da reprodução

Em 20/10/2026, João Correa, que não escreveu o código das seções, seguiu os quatro passos acima a partir de um clone limpo: ambiente novo com pip install -r requirements.txt e execução com jupyter nbconvert --execute, em Linux (WSL2), Python 3.13.5, pandas 2.3.3, requests 2.34.2, PySpark 4.0.3 e Java 21. Resultado: 47 células de código em sequência, sem erro, em 14 s; df_bruto com 2.800 linhas × 102 colunas; dados/preparados/faixas_preparadas.csv regravado idêntico ao versionado; três lotes de 30 eventos processados (90 reproduções); python scripts/verificar_entrega.py sem pendência no notebook.

RA1 / 07

Referências e anexos

Reúna as fontes utilizadas e os materiais complementares que ajudam a verificar a entrega.

Referências

DEEZER. *Deezer API*: documentação e API pública (/infos, /genre, /chart/{id}/tracks, /track/{id}, /album/{id}, /artist/{id}). 2026. Disponível em: https://developers.deezer.com/api e https://api.deezer.com. Acesso em: 16/10/2026.

ALVES, Rodrigo et al. *DataAudio*: app Flutter que consome a API da Deezer. 2026. Disponível em: https://github.com/rdsalvesPUC/dataaudio. Acesso em: 16/10/2026.

PYTHON SOFTWARE FOUNDATION. *Python 3.13 documentation*. Versão 3.13. Disponível em: https://docs.python.org/3.13/. Acesso em: 16/10/2026.

PANDAS DEVELOPMENT TEAM. *pandas documentation*. Versão 2.3. Disponível em: https://pandas.pydata.org/docs/. Acesso em: 16/10/2026.

REITZ, Kenneth. *Requests: HTTP for Humans*. Versão 2.34. Disponível em: https://requests.readthedocs.io/. Acesso em: 16/10/2026.

PUCPR. Data Analytics. *PjBL – RA1* (enunciado) e *Template_RA1_Data_Analytics.docx*. 2026. Disponível em: Canvas PUCPR (acesso restrito). Acesso em: 15/10/2026.

PUCPR. Data Analytics. Material de aula: *Data Ingestion and Data Quality*, *From Data Preparation to Data Transformation* e *Data Transformation*. 2026. Disponível em: Canvas PUCPR (acesso restrito). Acesso em: 15/10/2026.

Anexos opcionais

Identificação	Conteúdo	Localização
Anexo A	Dicionário do conjunto preparado	dados/preparados/ dicionario-faixas.md
Anexo B	Metadados da coleta de 16/10/2026	dados/brutos/2026-10-16/ metadados.json

Conferência da entrega

☒ Problema, recorte e perguntas analíticas estão definidos.

☒ Fontes e procedimentos de ingestão estão documentados.

☒ A qualidade foi avaliada com evidências verificáveis.

☒ As decisões de limpeza estão justificadas e validadas.

☒ O conjunto de dados preparado está identificado.

☒ O repositório contém instruções suficientes para reprodução.

☒ Os campos-modelo foram preenchidos e os exemplos revisados.

Identificação do projeto

Título do projeto: Perfil das faixas populares nos charts da Deezer por gênero
Integrantes do grupo: Eduardo Fabri, João Correa, Rodrigo Alves e João Pedro Cardoso
Descrição breve do problema ou contexto: os charts da Deezer dizem quais faixas estão no topo de cada gênero, mas não o que elas têm em comum. O projeto coleta esses charts pela API pública da Deezer e prepara os dados para comparar, entre gêneros, a duração das faixas, a idade do lançamento, a popularidade do artista e a concentração de artistas no ranking.

Preencha este notebook ao longo dos dois RAs. Registre decisões e interpretações em poucas frases e acrescente as células de código necessárias em cada etapa.

Preparação do ambiente

Esta célula prepara as bibliotecas, as pastas do projeto e a origem dos dados. No Google Colab, ela clona o repositório para ter acesso ao retrato bruto da API já coletado.

In [1]:
```
# Preparação do ambiente: bibliotecas, pastas do projeto e origem dos dados
import subprocess
import sys
from pathlib import Path

import pandas as pd

URL_REPOSITORIO = "https://github.com/Jcliz/data-analytics.git"
RAMO = "main"  # no Colab, troque pelo ramo do repositório que quiser executar
NO_COLAB = "google.colab" in sys.modules

if NO_COLAB:
    if Path("data-analytics").exists():
        # Clone de uma execução anterior nesta sessão do Colab: traz a versão mais nova do ramo
        subprocess.run(["git", "-C", "data-analytics", "pull", "-q", "origin", RAMO], check=False)
    else:
        subprocess.run(["git", "clone", "-q", "-b", RAMO, URL_REPOSITORIO], check=True)

# A raiz é a pasta que contém o CONTEXT.md (funciona no Colab, na raiz ou dentro de notebook/)
candidatos = [Path("data-analytics"), Path.cwd(), Path.cwd().parent]
```

```
RAIZ = next((p.resolve() for p in candidatos if (p /
"CONTEXT.md").exists()), None)
if RAIZ is None:
    raise FileNotFoundError(
        "Raiz do repositório não encontrada (pasta com o CONTEXT.md). Rode
o notebook de dentro do clone "
        f"(raiz ou notebook/) ou, no Colab, confira se o ramo '{RAMO}' já
tem a estrutura do projeto."
    )

PASTA_BRUTOS = RAIZ / "dados" / "brutos"
PASTA_PREPARADOS = RAIZ / "dados" / "preparados"
PASTA_STREAMING = RAIZ / "dados" / "streaming"
for pasta in (PASTA_BRUTOS, PASTA_PREPARADOS, PASTA_STREAMING):
    pasta.mkdir(parents=True, exist_ok=True)

# False: lê o retrato bruto versionado (ADR-0004). True: coleta de novo na
API da Deezer.
BAIXAR_DA_API = False

pd.set_option("display.max_columns", 30)
pd.set_option("display.width", 120)
print(f"Raiz do projeto: {RAIZ.name}")  # só o nome da pasta: o caminho
completo é da máquina de quem executa
print(f"Ambiente: {'Google Colab' if NO_COLAB else 'local'} · Python
{sys.version.split()[0]} · pandas {pd.__version__}")
```

```
Raiz do projeto: data-analytics
Ambiente: local · Python 3.13.5 · pandas 2.3.3
```

RA1 — Data Ingestion, Cleaning and Initial Analysis

Nesta etapa, carregue, prepare e explore os dados do projeto.

1. Contexto e objetivo

Apresente brevemente o problema investigado e o que o grupo pretende descobrir com os dados.

Resposta do grupo:

Os charts da Deezer mostram as faixas mais ouvidas em cada gênero, mas não o que elas têm em comum. O grupo quer descrever o perfil dessas faixas e ver se ele muda entre gêneros. O recorte é um único retrato dos charts da Deezer, o chart geral e o de cada gênero, na data da coleta registrada na seção 2. A unidade de observação é a faixa. As perguntas são:

- **P1:** como a duração das faixas varia entre os gêneros;
- **P2:** que parte das faixas de cada gênero foi lançada nos 12 meses anteriores à coleta;
- **P3:** se faixas de artistas com mais fãs têm `rank` mais alto, no geral e dentro de cada gênero;
- **P4:** quantos artistas distintos ocupam cada chart de gênero e que fatia das posições fica com os 5 artistas com mais faixas.

O detalhe do recorte e das lacunas está na seção 01 do documento do RA1.

2. Fonte e ingestão dos dados

- Identifique a fonte dos dados, incluindo link ou referência de acesso.
- Descreva brevemente o dataset e o que seus registros representam.
- Explique como os dados foram carregados e implemente a ingestão.

Resposta do grupo:

A fonte é a API pública da Deezer (`https://api.deezer.com` , documentação em `https://developers.deezer.com/api`), acessada sem autenticação e sem chave. Coletamos os charts de faixas mais tocadas: o chart geral e um chart por gênero da lista `/genre` . Cada registro do `df_bruto` é um item de um chart, ou seja, uma faixa na posição em que apareceu em um chart de origem; a mesma faixa pode aparecer em mais de um chart, e essas repetições ficam no bruto. Cada item recebe os campos extras da faixa (`/track/{id}` , como `bpm` , `isrc` e `release_date`), do álbum (`/album/{id}` , como gênero, gravadora e data de lançamento) e do artista (`/artist/{id}` , como `nb_fan` e `nb_album`), juntados pelo id.

A ingestão é feita por código nas células abaixo: as chamadas passam por uma função com pausa entre requisições, novas tentativas e tratamento do objeto `error` , que a API devolve mesmo com HTTP 200. Os charts são paginados por `index` e `limit` . As respostas são gravadas cruas no retrato bruto `dados/brutos/AAAA-MM-DD/` , com um `metadados.json` da coleta, e o notebook lê esse retrato por padrão (`BAIXAR_DA_API = False`), para que os números sejam os mesmos em toda execução.

O retrato usado é o de 16/10/2026 (país aparente US): 6.591 chamadas, sem erros, 28 charts de 100 itens (o geral e 27 gêneros). O `df_bruto` tem 2.800 linhas e 102 colunas, com 2.581 faixas, 2.292 álbuns e 1.688 artistas únicos, todos com o detalhe juntado.

In [2]:
```
# Parâmetros da coleta e acesso à API
import json
```

```
import platform
import time
from datetime import datetime, timezone

import requests
from IPython.display import display

URL_API = "https://api.deezer.com"
LIMITE_PAGINA = 100       # itens por página do chart (parâmetro limit)
TETO_POR_CHART = 300      # máximo de itens lidos por chart, mesmo que a
API ofereça mais
PAUSA_S = 0.15            # pausa entre chamadas; a API aceita cerca de 50
chamadas a cada 5 s
TENTATIVAS = 4            # tentativas por chamada antes de registrar o
erro
TIMEOUT_S = 20

sessao = requests.Session()
registro = {"chamadas": 0, "chamadas_por_endpoint": {}, "novas_tentativas":
0, "erros": []}


def chamar_api(caminho, endpoint, **params):
    """GET em URL_API + caminho. Devolve o JSON da resposta ou None,
registrando o erro.

    A Deezer devolve falhas como {"error": {...}} com HTTP 200: o código 4
(limite de chamadas)
    e falhas de rede ganham nova tentativa com espera crescente; os demais
erros são registrados.
    """
    ultimo_erro = None
    for tentativa in range(1, TENTATIVAS + 1):
        time.sleep(PAUSA_S)
        registro["chamadas"] += 1
        registro["chamadas_por_endpoint"][endpoint] =
registro["chamadas_por_endpoint"].get(endpoint, 0) + 1
        try:
            resposta = sessao.get(URL_API + caminho, params=params,
timeout=TIMEOUT_S)
            resposta.raise_for_status()
            corpo = resposta.json()
        except (requests.RequestException, ValueError) as erro:
            ultimo_erro = {"tipo": type(erro).__name__, "mensagem":
str(erro)[:200]}
        else:
            if not (isinstance(corpo, dict) and "error" in corpo):
                return corpo
            ultimo_erro = {"tipo": corpo["error"].get("type"), "codigo":
corpo["error"].get("code"),
                           "mensagem": corpo["error"].get("message")}
            if ultimo_erro["codigo"] != 4:   # objeto inexistente ou
parâmetro inválido: repetir não resolve
                break
        if tentativa < TENTATIVAS:
            registro["novas_tentativas"] += 1
```

```
            time.sleep(2 ** tentativa)
    registro["erros"].append({"caminho": caminho, "parametros": params,
**ultimo_erro})
    return None


print(f"API: {URL_API} · limit={LIMITE_PAGINA} · teto por chart=
{TETO_POR_CHART} · pausa={PAUSA_S} s")
print(f"Coletar de novo na API: {BAIXAR_DA_API}")
```

```
API: https://api.deezer.com · limit=100 · teto por chart=300 · pausa=0.15 s
Coletar de novo na API: False
```

In [3]:

```
# Coleta dos charts por gênero
if BAIXAR_DA_API:
    inicio_coleta = datetime.now(timezone.utc)
    infos = chamar_api("/infos", "/infos")   # país de onde a API enxerga a
requisição

    resposta_generos = chamar_api("/genre", "/genre")
    if resposta_generos is None:
        raise RuntimeError(f"A API não devolveu a lista de gêneros:
{registro['erros'][-1]}")
    generos = {g["id"]: g["name"] for g in resposta_generos["data"]}
    generos.setdefault(0, "All")   # chart geral

    charts = []
    for id_genero, nome_genero in generos.items():
        paginas = []
        for index in range(0, TETO_POR_CHART, LIMITE_PAGINA):
            pagina = chamar_api(f"/chart/{id_genero}/tracks",
"/chart/{id}/tracks",
                                index=index, limit=LIMITE_PAGINA)
            if pagina is None:
                break
            paginas.append({"index": index, "limit": LIMITE_PAGINA,
"resposta": pagina})
            lidos = len(pagina.get("data", []))
            if lidos < LIMITE_PAGINA or index + lidos >=
pagina.get("total", index + lidos):
                break   # última página do chart
        charts.append({"id_genero": id_genero, "nome_genero": nome_genero,
"paginas": paginas})
        itens = sum(len(p["resposta"].get("data", [])) for p in paginas)
        print(f"Chart {id_genero:>4} ({nome_genero}): {itens} itens em
{len(paginas)} página(s)")
    print(f"{len(charts)} charts coletados em {registro['chamadas']}
chamadas")
```

In [4]:

```
# Coleta dos detalhes de faixas, álbuns e artistas
if BAIXAR_DA_API:
    itens_chart = [item for c in charts for p in c["paginas"] for item in
p["resposta"].get("data", [])]
    ids_faixas = sorted({item["id"] for item in itens_chart})
    ids_albuns = sorted({item["album"]["id"] for item in itens_chart if
item.get("album")})
```

```
    ids_artistas = sorted({item["artist"]["id"] for item in itens_chart if
item.get("artist")})
    print(f"{len(itens_chart)} itens nos charts · {len(ids_faixas)} faixas
únicas · {len(ids_albuns)} álbuns únicos"
          f" · {len(ids_artistas)} artistas únicos")

    faixas = {}
    for i, id_faixa in enumerate(ids_faixas, 1):
        faixas[str(id_faixa)] = chamar_api(f"/track/{id_faixa}",
"/track/{id}")
        if i % 250 == 0:
            print(f"  faixas: {i}/{len(ids_faixas)}")
    albuns = {}
    for i, id_album in enumerate(ids_albuns, 1):
        albuns[str(id_album)] = chamar_api(f"/album/{id_album}",
"/album/{id}")
        if i % 250 == 0:
            print(f"  álbuns: {i}/{len(ids_albuns)}")
    artistas = {}
    for i, id_artista in enumerate(ids_artistas, 1):
        artistas[str(id_artista)] = chamar_api(f"/artist/{id_artista}",
"/artist/{id}")
        if i % 250 == 0:
            print(f"  artistas: {i}/{len(ids_artistas)}")
    fim_coleta = datetime.now(timezone.utc)
    print(f"Coleta terminada: {registro['chamadas']} chamadas,
{len(registro['erros'])} erro(s)")
```

In [5]:

```
# Gravação do retrato bruto
if BAIXAR_DA_API:
    pasta_retrato = PASTA_BRUTOS / inicio_coleta.strftime("%Y-%m-%d")
    pasta_retrato.mkdir(parents=True, exist_ok=True)
    conteudo = {"infos.json": infos, "generos.json": resposta_generos,
"charts.json": charts,
                "faixas.json": faixas, "albuns.json": albuns,
"artistas.json": artistas}
    for nome, dados in conteudo.items():
        # Respostas cruas da API, sem nenhuma alteração; JSON compacto para
o retrato caber no repositório
        (pasta_retrato / nome).write_text(json.dumps(dados,
ensure_ascii=False, separators=(",", ":")),
                                          encoding="utf-8")
    metadados = {
        "inicio_utc": inicio_coleta.isoformat(timespec="seconds"),
        "fim_utc": fim_coleta.isoformat(timespec="seconds"),
        "url_api": URL_API,
        "pais_aparente": (infos or {}).get("country_iso"),
        "endpoints": ["/infos", "/genre", "/chart/{id}/tracks",
"/track/{id}", "/album/{id}", "/artist/{id}"],
        "parametros": {"limit": LIMITE_PAGINA, "teto_por_chart":
TETO_POR_CHART, "pausa_s": PAUSA_S,
                       "tentativas": TENTATIVAS, "timeout_s": TIMEOUT_S},
        "chamadas": registro["chamadas"],
        "chamadas_por_endpoint": registro["chamadas_por_endpoint"],
        "novas_tentativas": registro["novas_tentativas"],
        "quantidade_erros": len(registro["erros"]),
```

```
        "erros": registro["erros"],
        "charts": len(charts),
        "itens_chart": len(itens_chart),
        "faixas_unicas": len(ids_faixas),
        "albuns_unicos": len(ids_albuns),
        "artistas_unicos": len(ids_artistas),
        "arquivos": sorted(conteudo) + ["metadados.json"],
        "ambiente": {"python": platform.python_version(), "pandas":
pd.__version__,
                     "requests": requests.__version__, "colab": NO_COLAB},
    }
    (pasta_retrato / "metadados.json").write_text(json.dumps(metadados,
ensure_ascii=False, indent=2),
                                                  encoding="utf-8")
    tamanho_mb = sum(a.stat().st_size for a in pasta_retrato.iterdir()) /
1e6
    print(f"Retrato gravado em {pasta_retrato.relative_to(RAIZ)}
({tamanho_mb:.1f} MB)")
```

In [6]:
```
# Leitura do retrato bruto mais recente
retratos = sorted(p for p in PASTA_BRUTOS.glob("????-??-??") if (p /
"metadados.json").exists())
if not retratos:
    raise FileNotFoundError(
        f"Nenhum retrato bruto em {PASTA_BRUTOS}. Atualize o clone do
repositório ou rode a seção 00b "
        "com BAIXAR_DA_API = True para coletar na API da Deezer."
    )
PASTA_RETRATO = retratos[-1]


def ler_json(nome):
    return json.loads((PASTA_RETRATO / nome).read_text(encoding="utf-8"))


metadados = ler_json("metadados.json")
charts = ler_json("charts.json")
faixas = ler_json("faixas.json")
albuns = ler_json("albuns.json")
artistas = ler_json("artistas.json")
print(f"Retrato lido: {PASTA_RETRATO.relative_to(RAIZ)} ({len(retratos)}
retrato(s) disponível(is))")
for arquivo in sorted(PASTA_RETRATO.iterdir()):
    print(f"  {arquivo.name:<15} {arquivo.stat().st_size / 1e6:6.2f} MB")
```

```
Retrato lido: dados/brutos/2026-10-16 (1 retrato(s) disponível(is))
  albuns.json      37.17 MB
  artistas.json     1.47 MB
  charts.json       5.65 MB
  faixas.json      13.35 MB
  generos.json      0.02 MB
  infos.json        0.00 MB
  metadados.json    0.00 MB
```

In [7]:
```
# Metadados do retrato lido
resumo_metadados = pd.Series({
```

```
    "Início da coleta (UTC)": metadados["inicio_utc"],
    "Fim da coleta (UTC)": metadados["fim_utc"],
    "País aparente da requisição": metadados["pais_aparente"],
    "Endpoints": ", ".join(metadados["endpoints"]),
    "Parâmetros": ", ".join(f"{k}={v}" for k, v in
metadados["parametros"].items()),
    "Chamadas à API": metadados["chamadas"],
    "Novas tentativas": metadados["novas_tentativas"],
    "Erros registrados": metadados["quantidade_erros"],
    "Charts coletados": metadados["charts"],
    "Itens nos charts": metadados["itens_chart"],
    "Faixas únicas": metadados["faixas_unicas"],
    "Álbuns únicos": metadados["albuns_unicos"],
    "Artistas únicos": metadados["artistas_unicos"],
    "Ambiente da coleta": ", ".join(f"{k} {v}" for k, v in
metadados["ambiente"].items()),
}, name="valor")
display(resumo_metadados.to_frame())
display(pd.Series(metadados["chamadas_por_endpoint"],
name="chamadas").to_frame())
if metadados["erros"]:
    display(pd.DataFrame(metadados["erros"]))
```

	valor
Início da coleta (UTC)	2026-10-16T21:03:16+00:00
Fim da coleta (UTC)	2026-10-16T21:41:09+00:00
País aparente da requisição	US
Endpoints	/infos, /genre, /chart/{id}/tracks, /track/{id...
Parâmetros	limit=100, teto_por_chart=300, pausa_s=0.15, t...
Chamadas à API	6591
Novas tentativas	0
Erros registrados	0
Charts coletados	28
Itens nos charts	2800
Faixas únicas	2581
Álbuns únicos	2292
Artistas únicos	1688
Ambiente da coleta	python 3.13.16, pandas 2.3.3, requests 2.34.2,...

	chamadas
/infos	1
/genre	1
/chart/{id}/tracks	28
/track/{id}	2581
/album/{id}	2292
/artist/{id}	1688

In [8]:
```
# Montagem do df_bruto
linhas = []
for chart in charts:
    ordem = 0
    for pagina in chart["paginas"]:
        for item in pagina["resposta"].get("data", []):
            ordem += 1
            linhas.append({"chart_id_genero": chart["id_genero"],
"chart_nome_genero": chart["nome_genero"],
                           "chart_ordem": ordem, **item})
itens = pd.json_normalize(linhas, sep="_")


def campos_simples(objeto, prefixo, ignorar=()):
    """Campos de primeiro nível de uma resposta (objetos aninhados ficam de
fora; listas ficam como vieram)."""
    return {f"{prefixo}{k}": v for k, v in objeto.items() if not
isinstance(v, dict) and k not in ignorar}


detalhe_faixas = pd.DataFrame([campos_simples(f, "detalhe_faixa_") for f in
faixas.values() if f])
detalhe_albuns = pd.DataFrame([
    {**campos_simples(a, "detalhe_album_", ignorar=("tracks",)),
     "detalhe_album_genres": (a.get("genres") or {}).get("data")}
    for a in albuns.values() if a
])
detalhe_artistas = pd.DataFrame([campos_simples(a, "detalhe_artista_") for
a in artistas.values() if a])

df_bruto = (
    itens
    .merge(detalhe_faixas, how="left", left_on="id",
right_on="detalhe_faixa_id")
    .merge(detalhe_albuns, how="left", left_on="album_id",
right_on="detalhe_album_id")
    .merge(detalhe_artistas, how="left", left_on="artist_id",
right_on="detalhe_artista_id")
)
print(f"df_bruto: {df_bruto.shape[0]} linhas × {df_bruto.shape[1]}
colunas")
```

```
df_bruto: 2800 linhas × 102 colunas
```

In [9]:
```
# Verificação do carregamento
origem_colunas = pd.Series(
    ["chart" if c.startswith("chart_") else "detalhe da faixa" if
c.startswith("detalhe_faixa_")
     else "detalhe do álbum" if c.startswith("detalhe_album_")
     else "detalhe do artista" if c.startswith("detalhe_artista_") else
"item do chart" for c in df_bruto.columns]
).value_counts().rename("colunas")
print(f"Linhas: {len(df_bruto)} (itens nos charts segundo os metadados:
{metadados['itens_chart']})")
print(f"Faixas únicas: {df_bruto['id'].nunique()} · álbuns únicos:
{df_bruto['album_id'].nunique()} · "
      f"artistas únicos: {df_bruto['artist_id'].nunique()}")
print(f"Itens sem detalhe da faixa:
{df_bruto['detalhe_faixa_id'].isna().sum()} · "
      f"sem detalhe do álbum: {df_bruto['detalhe_album_id'].isna().sum()} ·
"
      f"sem detalhe do artista:
{df_bruto['detalhe_artista_id'].isna().sum()}")
display(origem_colunas.to_frame())
display(df_bruto.groupby(["chart_id_genero",
"chart_nome_genero"]).size().rename("itens").to_frame())
display(df_bruto.head())
```

```
Linhas: 2800 (itens nos charts segundo os metadados: 2800)
Faixas únicas: 2581 · álbuns únicos: 2292 · artistas únicos: 1688
Itens sem detalhe da faixa: 0 · sem detalhe do álbum: 0 · sem detalhe do art
ista: 0
```

	colunas
item do chart	35
detalhe do álbum	26
detalhe da faixa	24
detalhe do artista	14
chart	3

		itens
chart_id_genero	chart_nome_genero	
0	All	100
2	African Music	100
16	Asian Music	100
65	Traditional Mexicano	100
67	Salsa	100
71	Cumbia	100
75	Brazilian Music	100
81	Indian Music	100
84	Country	100
85	Alternative	100
95	Kids	100
98	Classical	100
106	Electro	100
113	Dance	100
116	Rap/Hip Hop	100
122	Reggaeton	100
129	Jazz	100
132	Pop	100
144	Reggae	100
152	Rock	100
153	Blues	100
165	R&B	100
169	Soul & Funk	100
173	Films/Games	100
186	Christian	100
197	Latin Music	100
464	Metal	100
466	Folk	100

	chart_id_genero	chart_nome_genero	chart_ordem	id	title	title
0	0	All	1	3818963601	Dracula (with JENNIE)	
1	0	All	2	3895241341	Boston	
2	0	All	3	3834003271	Be Her	
3	0	All	4	3602074142	Choosin' Texas	C
4	0	All	5	4167204822	Been By Now	E

5 rows × 102 columns

3. Exploração inicial dos dados

Investigue a quantidade de registros e colunas, os tipos de dados, os valores ausentes e outras características relevantes da base.

Uma linha representa uma ocorrência de faixa em um chart no retrato da coleta. A chave candidata é o par `chart_id_genero` e `id`, dentro desse retrato. `id` identifica a faixa, mas não é único entre charts. As verificações abaixo contam ocorrências, não faixas distintas, salvo indicação explícita.

In [10]:
```
# Visão geral e tipos observados
import json
import numpy as np
import pandas as pd
from IPython.display import display

numero_linhas = len(df_bruto)
if numero_linhas == 0:
    raise ValueError("O diagnóstico exige um df_bruto não vazio; execute a
seção 2.")
print(f"Retrato: {numero_linhas} linhas × {df_bruto.shape[1]} colunas; "
      f"{df_bruto['id'].nunique()} faixas;
{df_bruto['chart_id_genero'].nunique()} charts.")
print(f"Memória, incluindo objetos:
{df_bruto.memory_usage(deep=True).sum():,} bytes.")
inventario =
df_bruto.dtypes.astype(str).rename("tipo_observado").to_frame()
display(inventario.reset_index(names="atributo"))
```

```
Retrato: 2800 linhas × 102 colunas; 2581 faixas; 28 charts.
Memória, incluindo objetos: 25,886,539 bytes.
```

	atributo	tipo_observado
0	chart_id_genero	int64
1	chart_nome_genero	object
2	chart_ordem	int64
3	id	int64
4	title	object
...	...	...
97	detalhe_artista_nb_album	int64
98	detalhe_artista_nb_fan	int64
99	detalhe_artista_radio	bool
100	detalhe_artista_tracklist	object
101	detalhe_artista_type	object

102 rows × 2 columns

In [11]:
```
# Ausências nulas e textos vazios
# isna() encontra nulos. Para textos, também contamos vazio e espaços como
ausência.
mascara_ausencias = df_bruto.isna()
textos_vazios = pd.DataFrame(False, index=df_bruto.index,
columns=df_bruto.columns)

for coluna in df_bruto.columns:
    if pd.api.types.is_string_dtype(df_bruto[coluna].dropna()):
        textos_vazios[coluna] =
df_bruto[coluna].str.strip().eq("").fillna(False)
        mascara_ausencias[coluna] = mascara_ausencias[coluna] |
textos_vazios[coluna]

ausencias = pd.DataFrame({
    "nulos": df_bruto.isna().sum(),
    "textos_vazios": textos_vazios.sum(),
    "total_ausentes": mascara_ausencias.sum(),
})
ausencias["percentual"] = 100 * ausencias["total_ausentes"] / numero_linhas
display(ausencias.reset_index(names="atributo"))
```

	atributo	nulos	textos_vazios	total_ausentes	percentual
0	chart_id_genero	0	0	0	0.0
1	chart_nome_genero	0	0	0	0.0
2	chart_ordem	0	0	0	0.0
3	id	0	0	0	0.0
4	title	0	0	0	0.0
...	...	...	...	...	...
97	detalhe_artista_nb_album	0	0	0	0.0
98	detalhe_artista_nb_fan	0	0	0	0.0
99	detalhe_artista_radio	0	0	0	0.0
100	detalhe_artista_tracklist	0	0	0	0.0
101	detalhe_artista_type	0	0	0	0.0

102 rows × 5 columns

In [12]:
```
# Duplicidade e limite da chave
# Algumas colunas guardam listas. Só para comparar linhas, transformamos
listas em texto.
# df_bruto continua intacto: a comparação usa outra tabela.
def texto_para_comparacao(valor):
    if isinstance(valor, (list, dict)):
        return json.dumps(valor, ensure_ascii=False, sort_keys=True)
    return valor

tabela_comparacao = df_bruto.copy()
for coluna in tabela_comparacao.columns:
    tabela_comparacao[coluna] =
tabela_comparacao[coluna].map(texto_para_comparacao)

duplicadas_inteiras = tabela_comparacao.duplicated()
duplicadas_id = df_bruto.duplicated("id")
duplicadas_chart = df_bruto.duplicated(["chart_id_genero", "id"])
print(f"Ocorrências excedentes por id:
{duplicadas_id.sum()}/{numero_linhas}; "
      f"por par chart-faixa: {duplicadas_chart.sum()}/{numero_linhas}; "
      f"linhas inteiras iguais:
{duplicadas_inteiras.sum()}/{numero_linhas}.")
```

```
Ocorrências excedentes por id: 219/2800; por par chart-faixa: 0/2800; linhas
inteiras iguais: 0/2800.
```

In [13]:
```
# Estatísticas descritivas e domínios numéricos
colunas_numericas = ["duration", "rank", "position", "detalhe_faixa_bpm",
                     "detalhe_faixa_gain", "detalhe_artista_nb_fan"]
numericos = df_bruto[colunas_numericas].apply(pd.to_numeric,
errors="coerce")
display(numericos.describe().T)
# São regras de integridade, não indicadores analíticos das perguntas P1 a
```

```
P4.
colunas_inteiras = ["id", "chart_id_genero", "chart_ordem", "duration",
"rank", "position",
                    "artist_id", "album_id", "detalhe_faixa_id",
"detalhe_album_id",
                    "detalhe_artista_id", "detalhe_artista_nb_fan"]
tipos_invalidos = pd.Series(False, index=df_bruto.index)
for coluna in colunas_inteiras:
    valores = pd.to_numeric(df_bruto[coluna], errors="coerce")
    tipos_invalidos |= valores.isna() | ~np.isfinite(valores) |
valores.mod(1).ne(0)
# Strings conversíveis também são divergências do tipo esperado, mesmo sem
perda na conversão.
for coluna in colunas_inteiras + ["detalhe_faixa_bpm",
"detalhe_faixa_gain"]:
    tipos_invalidos |= ~df_bruto[coluna].map(
        lambda valor: isinstance(valor, (int, float, np.integer,
np.floating))
        and not isinstance(valor, (bool, np.bool_))
    )
tipos_invalidos |= ~df_bruto["explicit_lyrics"].map(lambda valor:
isinstance(valor, (bool, np.bool_)))
# Cada comparação devolve True nas linhas com problema.
invalidos_numericos = numericos.isna().any(axis=1) |
~np.isfinite(numericos).all(axis=1)
invalidos_numericos = invalidos_numericos | (numericos["duration"] <= 0)
invalidos_numericos = invalidos_numericos | (numericos["position"] <= 0)
invalidos_numericos = invalidos_numericos | (numericos["rank"] < 0)
invalidos_numericos = invalidos_numericos |
(numericos["detalhe_artista_nb_fan"] < 0)
invalidos_numericos = invalidos_numericos | (numericos["detalhe_faixa_bpm"]
< 0)
for coluna in ["id", "artist_id", "album_id"]:
    valores = pd.to_numeric(df_bruto[coluna], errors="coerce")
    invalidos_numericos = invalidos_numericos | (valores <= 0)
bpm_zero = numericos["detalhe_faixa_bpm"] == 0
gain_zero = numericos["detalhe_faixa_gain"] == 0
# gain é um ajuste em dB: negativo é permitido; zero é sinalizado para
avaliação, não prova de erro.
```

	count	mean	std	min	25%	
duration	2800.0	2.284300e+02	9.743114e+01	41.0	179.00	
rank	2800.0	4.480146e+05	2.715404e+05	807.0	215125.00	4
position	2800.0	5.050000e+01	2.887123e+01	1.0	25.75	
detalhe_faixa_bpm	2800.0	4.285188e+01	6.408310e+01	0.0	0.00	
detalhe_faixa_gain	2800.0	-1.053176e+01	3.643281e+00	-30.7	-12.30	
detalhe_artista_nb_fan	2800.0	1.188450e+06	2.872902e+06	0.0	5530.00	

In [14]:
```
# Datas, formatos e categorias
colunas_datas = ["detalhe_faixa_release_date",
"detalhe_album_release_date"]
```

```
# A data de comparação vem dos metadados da coleta, para repetir o mesmo
resultado depois.
data_coleta = pd.Timestamp(metadados["inicio_utc"]).date()
datas_invalidas = pd.Series(False, index=df_bruto.index)
for coluna in colunas_datas:
    texto = df_bruto[coluna].astype("string")
    formato_correto = texto.str.fullmatch(r"\d{4}-\d{2}-
\d{2}").fillna(False)
    datas = pd.to_datetime(texto, format="%Y-%m-%d", errors="coerce")
    problema = ~formato_correto | datas.isna()
    problema = problema | (datas < pd.Timestamp("1900-01-01"))
    problema = problema | (datas > pd.Timestamp(data_coleta))
    datas_invalidas = datas_invalidas | problema

espacos = pd.Series(False, index=df_bruto.index)
for coluna in ["title", "artist_name", "chart_nome_genero"]:
    texto = df_bruto[coluna].astype("string")
    espacos = espacos | texto.ne(texto.str.strip()).fillna(False)
nomes_por_artista = df_bruto.groupby("artist_id")["artist_name"].nunique()
variacoes_artista =
df_bruto["artist_id"].isin(nomes_por_artista[nomes_por_artista.gt(1)].index
)
nomes_por_genero = df_bruto.groupby("chart_id_genero")
["chart_nome_genero"].nunique()
categorias_inconsistentes =
df_bruto["chart_id_genero"].isin(nomes_por_genero[nomes_por_genero.gt(1)].i
ndex)
nomes_normalizados =
df_bruto["chart_nome_genero"].astype("string").str.strip().str.casefold()
ids_por_nome = df_bruto.groupby(nomes_normalizados)
["chart_id_genero"].nunique()
categorias_inconsistentes |=
nomes_normalizados.isin(ids_por_nome[ids_por_nome.gt(1)].index)
chart_geral = df_bruto["chart_id_genero"].eq(0)
genero_album_generico = pd.to_numeric(df_bruto["detalhe_album_genre_id"],
errors="coerce").le(0)
```

In [15]:

```
# Possíveis outliers e integridade do enriquecimento
# Limites IQR são sinais para inspeção; extremos não são removidos.
limites_outliers = []
for coluna in ["duration", "rank"]:
    primeiro_quartil, terceiro_quartil = numericos[coluna].quantile([0.25,
0.75])
    amplitude = terceiro_quartil - primeiro_quartil
    inferior, superior = primeiro_quartil - 1.5 * amplitude,
terceiro_quartil + 1.5 * amplitude
    mascara = numericos[coluna].lt(inferior) |
numericos[coluna].gt(superior)
    limites_outliers.append({"atributo": coluna, "inferior": inferior,
"superior": superior,
                             "ocorrencias": int(mascara.sum()),
"denominador": numero_linhas})
    if coluna == "duration":
        outliers_duracao = mascara
    else:
        outliers_rank = mascara
```

```
display(pd.DataFrame(limites_outliers))
display(df_bruto.loc[outliers_duracao, ["id", "title", "duration",
"chart_nome_genero"]])
integridade = pd.Series(False, index=df_bruto.index)
for origem, detalhe in [("id", "detalhe_faixa_id"), ("album_id",
"detalhe_album_id"),
                        ("artist_id", "detalhe_artista_id")]:
    integridade |= df_bruto[origem].isna() | df_bruto[detalhe].isna() |
df_bruto[origem].ne(df_bruto[detalhe])
divergencia_duracao =
df_bruto["duration"].ne(df_bruto["detalhe_faixa_duration"])
divergencia_rank = df_bruto["rank"].ne(df_bruto["detalhe_faixa_rank"])
```

	atributo	inferior	superior	ocorrencias	denominador
0	duration	66.50	366.50	143	2800
1	rank	-445721.75	1316536.25	0	2800

	id	title	duration	chart_nome_genero
139	374283061	Purple Rain (2015 Paisley Park Remaster)	524	Pop
202	131391760	Cinderella (feat. Ty Dolla $ign)	480	Rap/Hip Hop
213	921331222	A Nightmare On My Street	368	Rap/Hip Hop
215	1580929572	The Humpty Dance	391	Rap/Hip Hop
230	6617606	C'mon N' Ride It (The Train)	451	Rap/Hip Hop
...	...	...	...	...
2646	29673331	Thunder & Gentle Rain Soundscape	599	Kids
2694	2023467557	PAW Patrol Opening Theme	41	Kids
2697	1678115237	Concerto in C Major for Flute, Harp and Orches...	519	Kids
2723	686087352	Merecumbe	381	Latin Music
2743	3171003001	BAILE INoLVIDABLE	367	Latin Music

143 rows × 4 columns

In [16]:
```
# Diagnóstico de qualidade Q1 a Q21
campos_essenciais = ["id", "chart_id_genero", "chart_nome_genero", "title",
"duration", "rank",
                     "position", "artist_id", "artist_name", "album_id",
"detalhe_faixa_release_date",
                     "detalhe_artista_nb_fan"]
verificacoes = [
    ("Q1", "Ausência em pelo menos um campo essencial às P1–P4 ou RA2",
mascara_ausencias[campos_essenciais].any(axis=1), "Ausências nulas e textos
vazios"),
    ("Q2", "Ausência de preview (nulo ou texto vazio)",
```

```
mascara_ausencias["preview"], "Ausências nulas e textos vazios"),
    ("Q3", "Ausência de title_version (nulo ou texto vazio)",
mascara_ausencias["title_version"], "Ausências nulas e textos vazios"),
    ("Q4", "Ocorrências excedentes por id, após a primeira", duplicadas_id,
"Duplicidade e limite da chave"),
    ("Q5", "Ocorrências excedentes pelo par chart_id_genero e id",
duplicadas_chart, "Duplicidade e limite da chave"),
    ("Q6", "Linhas inteiras repetidas, após a primeira",
duplicadas_inteiras, "Duplicidade e limite da chave"),
    ("Q7", "Tipo divergente de numérico inteiro/real ou booleano esperado",
tipos_invalidos, "Estatísticas descritivas e domínios numéricos"),
    ("Q8", "Numéricos não finitos/ausentes ou fora dos domínios descritos",
invalidos_numericos, "Estatísticas descritivas e domínios numéricos"),
    ("Q9", "bpm igual a zero (sinal de dado não informado)", bpm_zero,
"Estatísticas descritivas e domínios numéricos"),
    ("Q10", "gain igual a zero (a avaliar, não erro comprovado)",
gain_zero, "Estatísticas descritivas e domínios numéricos"),
    ("Q11", "Data fora de AAAA-MM-DD, inválida ou fora de 1900 até a
coleta", datas_invalidas, "Datas, formatos e categorias"),
    ("Q12", "Espaços nas pontas de título, artista ou gênero", espacos,
"Datas, formatos e categorias"),
    ("Q13", "Mais de um nome para um id de artista", variacoes_artista,
"Datas, formatos e categorias"),
    ("Q14", "Gênero com mais de um nome ou nome normalizado com mais de um
id", categorias_inconsistentes, "Datas, formatos e categorias"),
    ("Q15", "Chart geral id 0: não representa um gênero específico",
chart_geral, "Datas, formatos e categorias"),
    ("Q16", "Gênero do álbum genérico (id menor ou igual a zero)",
genero_album_generico, "Datas, formatos e categorias"),
    ("Q17", "Duração fora de Q1 − 1,5 IQR a Q3 + 1,5 IQR",
outliers_duracao, "Possíveis outliers e integridade do enriquecimento"),
    ("Q18", "Rank fora de Q1 − 1,5 IQR a Q3 + 1,5 IQR", outliers_rank,
"Possíveis outliers e integridade do enriquecimento"),
    ("Q19", "Id ausente ou divergente na junção de faixa/álbum/artista",
integridade, "Possíveis outliers e integridade do enriquecimento"),
    ("Q20", "Duração do chart diferente da duração em /track",
divergencia_duracao, "Possíveis outliers e integridade do enriquecimento"),
    ("Q21", "Rank do chart diferente do rank em /track", divergencia_rank,
"Possíveis outliers e integridade do enriquecimento"),
]
# Somar uma coluna de True/False conta quantas linhas apresentaram o
problema.
resultados = []
for identificador, regra, mascara, celula in verificacoes:
    quantidade = int(mascara.sum())
    resultados.append({
        "ID": identificador,
        "regra": regra,
        "quantidade": quantidade,
        "denominador": numero_linhas,
        "percentual": 100 * quantidade / numero_linhas,
        "celula": celula,
    })

diagnostico = pd.DataFrame(resultados)
display(diagnostico)
```

```
print("As proporções usam ocorrências nos charts e podem se sobrepor; não devem ser somadas.")
```

	ID	regra	quantidade	denominador	percentual	celula
0	Q1	Ausência em pelo menos um campo essencial às P...	0	2800	0.000000	Ausências nulas e textos vazios
1	Q2	Ausência de preview (nulo ou texto vazio)	1	2800	0.035714	Ausências nulas e textos vazios
2	Q3	Ausência de title_version (nulo ou texto vazio)	2533	2800	90.464286	Ausências nulas e textos vazios
3	Q4	Ocorrências excedentes por id, após a primeira	219	2800	7.821429	Duplicidade e limite da chave
4	Q5	Ocorrências excedentes pelo par chart_id_gener...	0	2800	0.000000	Duplicidade e limite da chave
5	Q6	Linhas inteiras repetidas, após a primeira	0	2800	0.000000	Duplicidade e limite da chave
6	Q7	Tipo divergente de numérico inteiro/real ou bo...	0	2800	0.000000	Estatísticas descritivas e domínios numéricos
7	Q8	Numéricos não finitos/ausentes ou fora dos dom...	0	2800	0.000000	Estatísticas descritivas e domínios numéricos
8	Q9	bpm igual a zero (sinal de dado não informado)	1890	2800	67.500000	Estatísticas descritivas e domínios numéricos
9	Q10	gain igual a zero (a avaliar, não erro comprov...	27	2800	0.964286	Estatísticas descritivas e domínios numéricos
10	Q11	Data fora de AAAA-MM-DD, inválida ou fora de 1...	1	2800	0.035714	Datas, formatos e categorias
11	Q12	Espaços nas pontas de título, artista ou gênero	1	2800	0.035714	Datas, formatos e categorias
12	Q13	Mais de um nome para um id de artista	0	2800	0.000000	Datas, formatos e categorias

	ID	regra	quantidade	denominador	percentual	celula
13	Q14	Gênero com mais de um nome ou nome normalizado...	0	2800	0.000000	Datas, formatos e categorias
14	Q15	Chart geral id 0: não representa um gênero esp...	100	2800	3.571429	Datas, formatos e categorias
15	Q16	Gênero do álbum genérico (id menor ou igual a ...	11	2800	0.392857	Datas, formatos e categorias
16	Q17	Duração fora de Q1 − 1,5 IQR a Q3 + 1,5 IQR	143	2800	5.107143	Possíveis outliers e integridade do enriquecim...
17	Q18	Rank fora de Q1 − 1,5 IQR a Q3 + 1,5 IQR	0	2800	0.000000	Possíveis outliers e integridade do enriquecim...
18	Q19	Id ausente ou divergente na junção de faixa/ál...	0	2800	0.000000	Possíveis outliers e integridade do enriquecim...
19	Q20	Duração do chart diferente da duração em /track	0	2800	0.000000	Possíveis outliers e integridade do enriquecim...
20	Q21	Rank do chart diferente do rank em /track	0	2800	0.000000	Possíveis outliers e integridade do enriquecim...

`As proporções usam ocorrências nos charts e podem se sobrepor; não devem ser somadas.`

Resposta do grupo:

No retrato de 16/10/2026, são 2.800 ocorrências, 102 colunas e 2.581 faixas. Há 219/2.800 repetições excedentes por `id` (7,82%), mas nenhuma pela chave chart-faixa. BPM é zero em 1.890/2.800 ocorrências (67,50%); `title_version` está ausente em 2.533/2.800 (90,46%). Uma ocorrência tem lançamento posterior à coleta, e 143/2.800 (5,11%) têm duração fora dos limites IQR. Evidências: células “Visão geral e tipos observados” e “Diagnóstico de qualidade Q1 a Q21”. A repetição por `id` exige preservar o chart de origem na

preparação, pois ele define os grupos das perguntas P1 a P4. O chart geral deve ser identificado separadamente dos gêneros. BPM zero e versões sem texto precisam de interpretação; valores extremos de duração são candidatos à inspeção, sem exclusão automática. O diagnóstico completo e as prioridades estão no documento “Compreensão e qualidade”, com as mesmas contagens do quadro Q1 a Q21.

4. Limpeza e preparação dos dados

Realize a limpeza necessária ao projeto. Verifique valores ausentes, duplicados e inconsistências e trate os problemas encontrados, quando necessário.

As decisões D1–D8 respondem às verificações Q1–Q21. A preparação preserva a participação de cada faixa nos charts, corrige espaços e sinaliza situações que exigem cuidado, sem excluir automaticamente dados por critérios estatísticos.

In [17]:
```
# Cópia de trabalho e referência do bruto
import numpy as np
import pandas as pd
from IPython.display import display

referencia_bruta_preparacao = df_bruto.copy(deep=True)
df_preparado = df_bruto.copy(deep=True)
total_antes_preparacao = len(df_bruto)
assert total_antes_preparacao > 0, "Execute a ingestão antes de preparar os dados."
print(f"Entrada: {total_antes_preparacao} ocorrências × {df_bruto.shape[1]} atributos.")
```

```
Entrada: 2800 ocorrências × 102 atributos.
```

In [18]:
```
# Preservação do vínculo entre faixa e chart
# D1 (Q4–Q6): a faixa pode se repetir entre charts, mas a chave composta é única.
chave_bruta = ["chart_id_genero", "id"]
repeticoes_id_antes = int(df_bruto.duplicated("id").sum())
duplicatas_chave_antes = int(df_bruto.duplicated(chave_bruta).sum())
assert df_bruto[chave_bruta].notna().all().all(), "Chave de origem incompleta."
assert duplicatas_chave_antes == 0, "Revise o diagnóstico: há pares chart-faixa repetidos."
pd.testing.assert_frame_equal(df_preparado[chave_bruta], df_bruto[chave_bruta])
print(f"D1: {repeticoes_id_antes}/{total_antes_preparacao} excedentes por id mantidos; "
      f"{duplicatas_chave_antes}/{total_antes_preparacao} duplicatas chart-faixa.")
```

```
D1: 219/2800 excedentes por id mantidos; 0/2800 duplicatas chart-faixa.
```

In [19]:
```
# Padronização dos campos textuais
# D2 (Q12): remover somente espaços nas pontas, preservando acentos e a
```

```
escrita.
colunas_textuais_preparacao = ["title", "artist_name", "chart_nome_genero"]
espacos_antes_preparacao = pd.Series(False, index=df_bruto.index)
alteracoes_textuais_preparacao = []
for coluna in colunas_textuais_preparacao:
    antes = df_preparado[coluna].astype("string")
    depois = antes.str.strip()
    alteradas = antes.ne(depois).fillna(False)
    espacos_antes_preparacao |= alteradas
    for indice in df_preparado.index[alteradas]:
        alteracoes_textuais_preparacao.append({
            "id_faixa": int(df_preparado.at[indice, "id"]),
            "campo": coluna, "antes": repr(antes.at[indice]),
            "depois": repr(depois.at[indice]),
        })
    df_preparado[coluna] = depois
    assert depois.eq(depois.str.strip()).fillna(True).all()
print(f"D2: {int(espacos_antes_preparacao.sum())}/{total_antes_preparacao}
"
      "ocorrências corrigidas; nenhuma linha excluída.")
display(pd.DataFrame(alteracoes_textuais_preparacao).head(10))
```

```
D2: 1/2800 ocorrências corrigidas; nenhuma linha excluída.
```

	id_faixa	campo	antes	depois
0	70057719	artist_name	'Sona Mohapatra\xa0'	'Sona Mohapatra'

In [20]:

```
# Conversão e validação temporal
# D3 (Q11): conservar o lançamento e identificar datas posteriores à
coleta.
texto_lancamento_preparacao =
df_preparado["detalhe_faixa_release_date"].astype("string")
assert texto_lancamento_preparacao.str.fullmatch(r"\d{4}-\d{2}-
\d{2}").fillna(False).all()
datas_lancamento_preparacao = pd.to_datetime(
    texto_lancamento_preparacao, format="%Y-%m-%d", errors="raise"
)
assert datas_lancamento_preparacao.notna().all()
assert datas_lancamento_preparacao.ge(pd.Timestamp("1900-01-01")).all()
assert datas_lancamento_preparacao.dt.strftime("%Y-%m-%d").eq(
    texto_lancamento_preparacao
).all()
data_retrato_preparacao = pd.Timestamp(metadados["inicio_utc"])
if data_retrato_preparacao.tzinfo is not None:
    data_retrato_preparacao =
data_retrato_preparacao.tz_convert("UTC").tz_localize(None)
data_retrato_preparacao = data_retrato_preparacao.normalize()
df_preparado["detalhe_faixa_release_date"] = datas_lancamento_preparacao
df_preparado["data_coleta"] = data_retrato_preparacao
df_preparado["lancamento_posterior_coleta"] = (
    datas_lancamento_preparacao > data_retrato_preparacao
)
print(f"D3: {int(df_preparado['lancamento_posterior_coleta'].sum())}/"
      f"{total_antes_preparacao} lançamentos posteriores à coleta,
preservados.")
```

```
display(df_preparado.loc[
    df_preparado["lancamento_posterior_coleta"],
    ["id", "title", "detalhe_faixa_release_date", "data_coleta"],
].head(10))
```

D3: 1/2800 lançamentos posteriores à coleta, preservados.

	id	title	detalhe_faixa_release_date	data_coleta
1755	4346251442	Dirty Bird	2026-10-28	2026-10-16

In [21]:
```
# Identificação do chart de origem
# D4 (Q14–Q16): o gênero do álbum não substitui o chart de origem.
df_preparado["tipo_chart"] = df_preparado["chart_id_genero"].map(
    lambda identificador: "geral" if identificador == 0 else "genero"
)
charts_gerais_preparacao = df_preparado["chart_id_genero"].eq(0)
assert
df_preparado["tipo_chart"].eq("geral").equals(charts_gerais_preparacao)
assert df_preparado.groupby("chart_id_genero")
["chart_nome_genero"].nunique().eq(1).all()
print(f"D4: {int(charts_gerais_preparacao.sum())}/{total_antes_preparacao}
"
      "ocorrências identificadas como chart geral, sem exclusão.")
```

D4: 100/2800 ocorrências identificadas como chart geral, sem exclusão.

In [22]:
```
# Sinalização dos extremos de duração
# D7 (Q17–Q18): reproduzir a regra da seção 3, sem alterar duração ou rank.
quartis_duracao_preparacao = df_bruto["duration"].quantile([0.25, 0.75])
amplitude_duracao_preparacao = quartis_duracao_preparacao.iloc[1] - (
    quartis_duracao_preparacao.iloc[0]
)
limite_inferior_duracao = quartis_duracao_preparacao.iloc[0] - (
    1.5 * amplitude_duracao_preparacao
)
limite_superior_duracao = quartis_duracao_preparacao.iloc[1] + (
    1.5 * amplitude_duracao_preparacao
)
df_preparado["duracao_fora_limites_iqr"] = (
    df_preparado["duration"].lt(limite_inferior_duracao)
    | df_preparado["duration"].gt(limite_superior_duracao)
)
pd.testing.assert_series_equal(df_preparado["duration"],
df_bruto["duration"])
pd.testing.assert_series_equal(df_preparado["rank"], df_bruto["rank"])
print(f"D7: limites {limite_inferior_duracao:g} a
{limite_superior_duracao:g} s; "
      f"{int(df_preparado['duracao_fora_limites_iqr'].sum())}/"
      f"{total_antes_preparacao} ocorrências sinalizadas e mantidas.")
```

D7: limites 66.5 a 366.5 s; 143/2800 ocorrências sinalizadas e mantidas.

In [23]:
```
# Registro das decisões de preparação
decisoes_preparacao = pd.DataFrame([
    ("D1", "Q4–Q6", "Preservar o vínculo chart-faixa", "Nenhuma linha
excluída"),
```

```
    ("D2", "Q12", "Remover espaços nas pontas", "Preservar a escrita dos
nomes"),
    ("D3", "Q11", "Converter e sinalizar datas", "Não imputar lançamento"),
    ("D4", "Q14–Q16", "Identificar o tipo de chart", "Preservar a origem"),
    ("D5", "Q2, Q3, Q9, Q10", "Selecionar atributos na seção 5", "Não
excluir faixas"),
    ("D6", "Q7, Q8", "Padronizar nomes e tipos na seção 5", "Não alterar
números válidos"),
    ("D7", "Q17, Q18", "Sinalizar extremos", "Não cortar ou substituir
valores"),
    ("D8", "Q1, Q13, Q19–Q21", "Revalidar consistência na seção 5", "Manter
dados aprovados"),
], columns=["decisao", "diagnostico", "acao", "justificativa"])
display(decisoes_preparacao)
pd.testing.assert_frame_equal(df_bruto, referencia_bruta_preparacao)
assert len(df_preparado) == total_antes_preparacao
```

	decisao	diagnostico	acao	justificativa
0	D1	Q4–Q6	Preservar o vínculo chart-faixa	Nenhuma linha excluída
1	D2	Q12	Remover espaços nas pontas	Preservar a escrita dos nomes
2	D3	Q11	Converter e sinalizar datas	Não imputar lançamento
3	D4	Q14–Q16	Identificar o tipo de chart	Preservar a origem
4	D5	Q2, Q3, Q9, Q10	Selecionar atributos na seção 5	Não excluir faixas
5	D6	Q7, Q8	Padronizar nomes e tipos na seção 5	Não alterar números válidos
6	D7	Q17, Q18	Sinalizar extremos	Não cortar ou substituir valores
7	D8	Q1, Q13, Q19–Q21	Revalidar consistência na seção 5	Manter dados aprovados

Resposta do grupo:

Preservamos as 2.800 ocorrências e o vínculo faixa-chart (D1), com 219/2.800 excedentes por id e nenhuma duplicata pela chave composta. Corrigimos espaços nas pontas em 1/2.800 ocorrências (D2), convertemos o lançamento e sinalizamos 1/2.800 datas posteriores à coleta (D3). Identificamos separadamente as 100/2.800 ocorrências do chart geral (D4). As 143/2.800 durações fora dos limites IQR foram sinalizadas, sem exclusão nem substituição (D7).

Evidências: células “Preservação do vínculo entre faixa e chart”, “Padronização dos campos textuais”, “Conversão e validação temporal”, “Identificação do chart de origem” e “Sinalização dos extremos de duração”. A seção 5 conclui D5, D6 e

D8 com seleção, padronização, validação e gravação. O bruto permanece intacto.

5. Transformação dos dados

Realize as transformações necessárias à análise, como criação de colunas, filtros, agrupamentos, conversões ou junções, quando aplicáveis.

Selecionamos os atributos, padronizamos nomes e tipos e gravamos o conjunto preparado. A unidade continua sendo uma ocorrência de faixa por chart e data de coleta. As perguntas analíticas serão respondidas posteriormente.

In [24]:
```
# Seleção e organização dos atributos
# D5: doze atributos do bruto e quatro campos de contexto ou qualidade.
nomes_colunas_preparadas = {
    "id": "id_faixa", "title": "titulo", "chart_id_genero": "id_chart",
    "chart_nome_genero": "genero", "tipo_chart": "tipo_chart",
    "position": "posicao_chart", "artist_id": "id_artista", "artist_name":
"artista",
    "album_id": "id_album", "duration": "duracao_s", "rank": "rank",
    "detalhe_artista_nb_fan": "numero_fas",
    "detalhe_faixa_release_date": "data_lancamento", "data_coleta":
"data_coleta",
    "lancamento_posterior_coleta": "lancamento_posterior_coleta",
    "duracao_fora_limites_iqr": "duracao_fora_limites_iqr",
}
df_preparado = df_preparado.loc[:, list(nomes_colunas_preparadas)].rename(
    columns=nomes_colunas_preparadas
)
assert df_preparado.columns.tolist() ==
list(nomes_colunas_preparadas.values())
assert len(df_preparado) == total_antes_preparacao
print(f"D5: {df_bruto.shape[1]} atributos de origem → "
      f"{df_preparado.shape[1]} atributos finais; {len(df_preparado)}
linhas preservadas.")
```

```
D5: 102 atributos de origem → 16 atributos finais; 2800 linhas preservadas.
```

In [25]:
```
# Padronização dos nomes e tipos
# D6: rank mantém o nome já esperado pelo gerador de eventos da seção 9.
colunas_inteiras_preparadas = [
    "id_faixa", "id_chart", "posicao_chart", "id_artista", "id_album",
    "duracao_s", "rank", "numero_fas",
]
colunas_textuais_preparadas = ["titulo", "genero", "tipo_chart", "artista"]
colunas_datas_preparadas = ["data_lancamento", "data_coleta"]
colunas_booleanas_preparadas = [
    "lancamento_posterior_coleta", "duracao_fora_limites_iqr",
]
for coluna in colunas_inteiras_preparadas:
    valores = pd.to_numeric(df_preparado[coluna], errors="raise")
    assert valores.notna().all() and np.isfinite(valores).all(), coluna
    assert valores.mod(1).eq(0).all(), f"Inteiro fracionário em {coluna}."
```

```
    df_preparado[coluna] = valores.astype("int64")
for coluna in colunas_textuais_preparadas:
    df_preparado[coluna] = df_preparado[coluna].astype("string")
for coluna in colunas_datas_preparadas:
    df_preparado[coluna] = pd.to_datetime(
        df_preparado[coluna], errors="raise"
    ).astype("datetime64[ns]")
for coluna in colunas_booleanas_preparadas:
    assert pd.api.types.is_bool_dtype(df_preparado[coluna]), coluna
display(df_preparado.dtypes.astype(str).rename("tipo").to_frame())
```

	tipo
id_faixa	int64
titulo	string
id_chart	int64
genero	string
tipo_chart	string
posicao_chart	int64
id_artista	int64
artista	string
id_album	int64
duracao_s	int64
rank	int64
numero_fas	int64
data_lancamento	datetime64[ns]
data_coleta	datetime64[ns]
lancamento_posterior_coleta	bool
duracao_fora_limites_iqr	bool

In [26]:
```
# Verificações finais do conjunto preparado
# D8: falhas são expostas; não há preenchimento automático nem remoção de
linhas.
assert df_preparado.notna().all().all(), "O conjunto selecionado tem
valores ausentes."
chave_preparada = ["data_coleta", "id_chart", "id_faixa"]
assert not df_preparado.duplicated(chave_preparada).any()
for coluna in ["id_faixa", "id_artista", "id_album", "duracao_s",
"posicao_chart"]:
    assert df_preparado[coluna].gt(0).all(), coluna
for coluna in ["id_chart", "rank", "numero_fas"]:
    assert df_preparado[coluna].ge(0).all(), coluna
for coluna in colunas_textuais_preparadas:
    assert df_preparado[coluna].str.strip().ne("").all(), coluna
    assert df_preparado[coluna].eq(df_preparado[coluna].str.strip()).all(),
coluna
```

```
assert df_preparado["tipo_chart"].isin(["geral", "genero"]).all()
assert
df_preparado["tipo_chart"].eq("geral").eq(df_preparado["id_chart"].eq(0)).a
ll()
assert df_preparado.groupby("id_chart")["genero"].nunique().eq(1).all()
assert df_preparado.groupby("id_artista")["artista"].nunique().eq(1).all()
assert df_preparado["lancamento_posterior_coleta"].equals(
    df_preparado["data_lancamento"].gt(df_preparado["data_coleta"])
)
assert df_preparado["duracao_fora_limites_iqr"].equals(
    df_preparado["duracao_s"].lt(limite_inferior_duracao)
    | df_preparado["duracao_s"].gt(limite_superior_duracao)
)
# A correspondência linha a linha garante que a seleção não perdeu a
origem.
for origem, destino in nomes_colunas_preparadas.items():
    if destino in colunas_inteiras_preparadas:
        pd.testing.assert_series_equal(
            df_preparado[destino], df_bruto[origem].astype("int64"),
check_names=False
        )
for origem, detalhe in [
    ("id", "detalhe_faixa_id"), ("album_id", "detalhe_album_id"),
    ("artist_id", "detalhe_artista_id"),
]:
    assert df_bruto[origem].eq(df_bruto[detalhe]).all(), detalhe
assert
df_preparado["duracao_s"].eq(df_bruto["detalhe_faixa_duration"]).all()
assert df_preparado["rank"].eq(df_bruto["detalhe_faixa_rank"]).all()
assert df_preparado["data_lancamento"].dt.strftime("%Y-%m-%d").eq(
    df_bruto["detalhe_faixa_release_date"].astype("string")
).all()
assert df_preparado["data_coleta"].eq(data_retrato_preparacao).all()
pd.testing.assert_frame_equal(df_bruto, referencia_bruta_preparacao)
print(f"Validações aprovadas em {len(df_preparado)} ocorrências: "
      "tipos, domínios, ausências, chave, datas, sinalizações e
integridade.")
```

```
Validações aprovadas em 2800 ocorrências: tipos, domínios, ausências, chave,
datas, sinalizações e integridade.
```

In [27]:
```
# Comparação antes e depois da preparação
espacos_depois_preparacao = pd.Series(False, index=df_preparado.index)
for coluna in colunas_textuais_preparadas:
    espacos_depois_preparacao |= df_preparado[coluna].ne(
        df_preparado[coluna].str.strip()
    )
comparacao_preparacao = pd.DataFrame([
    ("Registros", total_antes_preparacao, len(df_preparado)),
    ("Atributos", df_bruto.shape[1], df_preparado.shape[1]),
    ("Ocorrências com espaços nas pontas",
int(espacos_antes_preparacao.sum()),
     int(espacos_depois_preparacao.sum())),
    ("Duplicatas chart-faixa", duplicatas_chave_antes,
     int(df_preparado.duplicated(chave_preparada).sum())),
    ("Excedentes por id da faixa", repeticoes_id_antes,
```

```
     int(df_preparado.duplicated("id_faixa").sum())),
    ("Ocorrências do chart geral", int(charts_gerais_preparacao.sum()),
     int(df_preparado["tipo_chart"].eq("geral").sum())),
    ("Lançamentos posteriores à coleta",
     int(datas_lancamento_preparacao.gt(data_retrato_preparacao).sum()),
     int(df_preparado["lancamento_posterior_coleta"].sum())),
    ("Durações fora dos limites IQR",
     int((df_bruto["duration"].lt(limite_inferior_duracao)
          | df_bruto["duration"].gt(limite_superior_duracao)).sum()),
     int(df_preparado["duracao_fora_limites_iqr"].sum())),
], columns=["medida", "antes", "depois"])
display(comparacao_preparacao)
print(f"Denominador das medidas por ocorrência: {total_antes_preparacao}
antes "
      f"e {len(df_preparado)} depois; as medidas podem se sobrepor.")
```

	medida	antes	depois
0	Registros	2800	2800
1	Atributos	102	16
2	Ocorrências com espaços nas pontas	1	0
3	Duplicatas chart-faixa	0	0
4	Excedentes por id da faixa	219	219
5	Ocorrências do chart geral	100	100
6	Lançamentos posteriores à coleta	1	1
7	Durações fora dos limites IQR	143	143

```
Denominador das medidas por ocorrência: 2800 antes e 2800 depois; as medidas
podem se sobrepor.
```

In [28]:
```
# Gravação e releitura do conjunto preparado
arquivo_preparado = PASTA_PREPARADOS / "faixas_preparadas.csv"
# Fim de linha "\n" fixo: o arquivo sai igual no Windows, no Linux e no
Colab
df_preparado.to_csv(arquivo_preparado, index=False, encoding="utf-8",
date_format="%Y-%m-%d",
                    lineterminator="\n")
tipos_releitura_preparacao = {
    **{coluna: "int64" for coluna in colunas_inteiras_preparadas},
    **{coluna: "string" for coluna in colunas_textuais_preparadas},
    **{coluna: "bool" for coluna in colunas_booleanas_preparadas},
}
releitura_preparacao = pd.read_csv(
    arquivo_preparado, encoding="utf-8", dtype=tipos_releitura_preparacao,
    parse_dates=colunas_datas_preparadas,
)
pd.testing.assert_frame_equal(df_preparado, releitura_preparacao)
tamanho_preparado = f"{arquivo_preparado.stat().st_size:,}".replace(",",
".")
print(f"Arquivo: {arquivo_preparado.relative_to(RAIZ)} · UTF-8, separador
vírgula, "
      f"{tamanho_preparado} bytes.")
```

```
print(f"Releitura validada: {len(releitura_preparacao)} linhas × "
      f"{releitura_preparacao.shape[1]} colunas, sem perda de valores.")
display(df_preparado.head(5))
```

```
Arquivo: dados/preparados/faixas_preparadas.csv · UTF-8, separador vírgula, 367.932 bytes.
Releitura validada: 2800 linhas × 16 colunas, sem perda de valores.
```

	id_faixa	titulo	id_chart	genero	tipo_chart	posicao_chart	id_artista
0	3818963601	Dracula (with JENNIE)	0	All	geral	1	134790
1	3895241341	Boston	0	All	geral	2	181157207
2	3834003271	Be Her	0	All	geral	3	14867123
3	3602074142	Choosin' Texas	0	All	geral	4	14867123
4	4167204822	Been By Now	0	All	geral	5	7188840

Resposta do grupo:

O conjunto preparado está em `dados/preparados/faixas_preparadas.csv`, com 2.800 ocorrências e 16 colunas: 12 atributos selecionados do bruto e quatro campos de contexto ou qualidade. Nenhuma linha foi excluída. Os espaços nas pontas passaram de 1/2.800 para 0/2.800 ocorrências. A chave data-chart-faixa permaneceu sem duplicatas; as 219/2.800 repetições por id foram preservadas. Evidências: células “Comparação antes e depois da preparação” e “Gravação e releitura do conjunto preparado”.

As 100/2.800 ocorrências do chart geral estão identificadas em `tipo_chart`. A data futura (1/2.800) e os extremos de duração (143/2.800) foram mantidos e sinalizados, sem confirmação de suas causas. O CSV não inclui BPM, ganho, prévia ou versão complementar; esses dados continuam no bruto. O gerador do RA2 precisa filtrar o chart geral ao usar o preparado, mantendo somente `tipo_chart == "genero"` para os eventos por gênero.

7. Considerações do RA1

Registre brevemente:

- O que o grupo descobriu sobre os dados.
- Quais dificuldades ou limitações encontrou.
- Como os dados preparados poderão ser utilizados nas próximas etapas.

Resposta do grupo:

O que descobrimos sobre os dados. O retrato de 16/10/2026 tem 2.800 ocorrências de 2.581 faixas em 28 charts, o geral e 27 gêneros. Nos campos que as perguntas usam, a qualidade de origem é boa: nenhuma ausência nos campos essenciais, nenhum tipo divergente e a mesma duração e o mesmo `rank` no chart e no detalhe da faixa nas 2.800 ocorrências (Q1, Q7, Q20 e Q21). Os problemas estão em campos fora das perguntas: `bpm` zerado em 1.890/2.800 ocorrências (67,50%) e `title_version` vazio em 2.533/2.800 (90,46%). Uma faixa pode estar em mais de um chart (219/2.800 ocorrências excedentes por `id`, 7,82%), por isso cada linha do conjunto preparado é uma faixa num chart. Evidência: seção 3, célula "Diagnóstico de qualidade Q1 a Q21".

Dificuldades e limitações. O chart muda todo dia e varia com o país: os números valem para um único retrato, coletado com país aparente US (seção 2). A API não informa reproduções, só o `rank`, um índice calculado pela Deezer. Uma data de lançamento posterior à coleta (1/2.800) e 143/2.800 durações fora dos limites IQR (5,11%) foram mantidas e sinalizadas, sem confirmar a causa. As 100/2.800 ocorrências do chart geral não representam um gênero. `bpm`, ganho, prévia e versão ficaram fora do conjunto preparado e continuam só no retrato bruto. Evidências: seção 4, células "Conversão e validação temporal", "Identificação do chart de origem" e "Sinalização dos extremos de duração".

Uso nas próximas etapas. O conjunto `dados/preparados/faixas_preparadas.csv` tem 2.800 linhas e 16 colunas, gravado e relido sem perda (seção 5, célula "Gravação e releitura do conjunto preparado"). No RA2, as 2.700 ocorrências dos charts de gênero alimentam os eventos de reprodução simulados (seção 9, célula "Faixas que alimentam os eventos"). No RA3, `duracao_s`, `data_lancamento`, `rank`, `numero_fas`, `id_artista`, `posicao_chart` e `genero` atendem às perguntas P1 a P4; `tipo_chart` separa o chart geral, e `lancamento_posterior_coleta` e `duracao_fora_limites_iqr` permitem decidir, na análise, como tratar a data futura e os extremos.

RA2 — Streaming Data Processing and Time Windows

Utilize os dados e o contexto do projeto para demonstrar o processamento em streaming e a aplicação de janelas temporais. Escolha um dos cenários:

- **Cenário A — Dados estáticos:** simule a chegada de novos registros utilizando dados artificiais coerentes com o contexto do projeto.

- **Cenário B — Dados em streaming:** utilize sua própria fonte de dados contínuos, sem necessidade de simulação.

8. Objetivo da análise em streaming

Responda brevemente:

- O que o grupo deseja monitorar ou analisar?
- Qual informação espera obter com o processamento em streaming?

Resposta do grupo:

No RA1, o grupo compara entre gêneros o perfil das faixas que estão no topo dos charts da Deezer. No RA2, o grupo quer **monitorar as reproduções dessas faixas por gênero ao longo do tempo**. Como a API da Deezer não oferece um fluxo de reproduções, usamos o cenário A: eventos de reprodução simulados, sorteados entre as faixas e os gêneros do conjunto preparado do RA1.

Com o processamento em streaming, esperamos saber, a cada janela de tempo, **quantas reproduções cada gênero acumula** e **qual gênero concentra mais reproduções naquele intervalo**, e ver esses números mudarem conforme novos lotes de eventos chegam. O tipo e a duração da janela estão na seção 10.

9. Fonte de dados e processamento

- Informe o cenário escolhido: dados reais em streaming ou simulação.
- Explique brevemente como os novos registros chegam.
- Implemente o processamento em tempo real.
- Demonstre o recebimento de novos registros.

Resposta do grupo:

Usamos o cenário A, de simulação, porque a API da Deezer não oferece um fluxo de reproduções em tempo real (ADR-0005). Cada registro novo é uma reprodução de faixa, com `id_evento`, `id_faixa`, `genero`, `artista`, `duracao_ouvida_s` e `instante`. A função `gerar_eventos` sorteia faixas e gêneros reais do conjunto do RA1, com chance proporcional ao `rank` da Deezer, semente 42 e primeiro instante em 21/10/2026 às 10:00, e espaça os eventos de 5 a 30 segundos.

Os registros chegam em lotes: `publicar_lote` grava cada lote como um arquivo JSON, com um evento por linha, e só então o move para `dados/streaming/entrada/`, a pasta que o Spark monitora com

`spark.readStream` e esquema explícito. A consulta de recebimento conta os eventos que chegaram: antes do 1º lote a tabela está vazia e, depois dele, mostra 30 eventos, de 10:00:28 a 10:09:05, de 19 gêneros. A janela temporal é aplicada na seção 10, e os lotes seguintes chegam na seção 11.

In [29]:
```
%%capture
# Instalação do PySpark (só no Colab)
if NO_COLAB:
    %pip install -q "pyspark==4.0.3"
```

In [30]:
```
# Sessão do Spark
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.types import IntegerType, LongType, StringType,
StructField, StructType, TimestampType

import os

# Os avisos de inicialização do Java e do Spark (nome da máquina, perfil de
log) vão para um arquivo,
# e não para a saída da célula; se a sessão não abrir, o fim desse arquivo
é mostrado com o erro.
LOG_SPARK = PASTA_STREAMING / "spark.log"
saida_erro = os.dup(2)
with open(LOG_SPARK, "w") as arquivo_log:
    os.dup2(arquivo_log.fileno(), 2)
    try:
        spark = (
            SparkSession.builder.master("local[2]")
            .appName("data-analytics — RA2")
            .config("spark.sql.session.timeZone", "America/Sao_Paulo")
            .config("spark.sql.shuffle.partitions", "2")
            .getOrCreate()
        )
    except Exception:
        os.dup2(saida_erro, 2)
        print(LOG_SPARK.read_text()[-2000:])
        raise
    finally:
        os.dup2(saida_erro, 2)
        os.close(saida_erro)
# Aplica o fuso também se já houver uma sessão aberta (reexecução da seção)
spark.conf.set("spark.sql.session.timeZone", "America/Sao_Paulo")
spark.sparkContext.setLogLevel("ERROR")

# Consultas que sobraram de uma execução anterior seguram arquivos das
pastas abaixo
for consulta_antiga in spark.streams.active:
    consulta_antiga.stop()

print(f"Spark {spark.version} · fuso
{spark.conf.get('spark.sql.session.timeZone')}")
```

```
Spark 4.0.3 · fuso America/Sao_Paulo
```

In [31]:
```
# Pastas do streaming
import json
import shutil
from datetime import datetime, timedelta
from uuid import uuid4

import numpy as np

PASTA_ENTRADA = PASTA_STREAMING / "entrada"          # monitorada pelo
Spark
PASTA_PUBLICACAO = PASTA_STREAMING / "publicacao"    # onde o lote é
escrito antes de ser publicado
PASTA_CHECKPOINT = PASTA_STREAMING / "checkpoint"    # progresso das
consultas
for pasta in (PASTA_ENTRADA, PASTA_PUBLICACAO, PASTA_CHECKPOINT):
    shutil.rmtree(pasta, ignore_errors=True)
    pasta.mkdir(parents=True)

print("Arquivos na entrada:", sorted(p.name for p in
PASTA_ENTRADA.iterdir()))
```

```
Arquivos na entrada: []
```

In [32]:
```
# Faixas que alimentam os eventos
COLUNAS_FONTE = {"id_faixa": "id_faixa", "genero": "genero", "artista":
"artista",
                 "duracao_s": "duracao_s", "rank": "rank"}

if "df_preparado" not in globals():
    arquivo_preparado = PASTA_PREPARADOS / "faixas_preparadas.csv"
    if arquivo_preparado.exists():
        df_preparado = pd.read_csv(arquivo_preparado)
if "df_preparado" in globals():
    faixas_fonte = (
        df_preparado[df_preparado["tipo_chart"] == "genero"]
        .rename(columns={v: k for k, v in COLUNAS_FONTE.items()})
[list(COLUNAS_FONTE)]
    )
    origem_faixas = "conjunto preparado do RA1, charts de gênero"
elif "df_bruto" in globals():
    faixas_fonte = (
        df_bruto[df_bruto["chart_id_genero"] != 0]
        .rename(columns={"id": "id_faixa", "chart_nome_genero": "genero",
"artist_name": "artista",
                         "duration": "duracao_s"})[list(COLUNAS_FONTE)]
    )
    origem_faixas = "df_bruto, charts de gênero (fonte provisória)"
else:
    raise FileNotFoundError(
        "Conjunto preparado não encontrado (df_preparado ou
dados/preparados/faixas_preparadas.csv). "
        "Execute as seções do RA1 antes da seção 9."
    )
faixas_fonte = faixas_fonte.dropna().reset_index(drop=True)

print(f"Origem: {origem_faixas}")
```

```
print(f"{len(faixas_fonte)} pares faixa-gênero · {faixas_fonte['id_faixa'].nunique()} faixas · "
      f"{faixas_fonte['genero'].nunique()} gêneros · {faixas_fonte['artista'].nunique()} artistas")
```

```
Origem: conjunto preparado do RA1, charts de gênero
2700 pares faixa-gênero · 2578 faixas · 27 gêneros · 1686 artistas
```

In [33]:
```
# Esquema do evento de reprodução
esquema_eventos = StructType([
    StructField("id_evento", IntegerType()),        # sequencial, único na simulação
    StructField("id_faixa", LongType()),            # id da Deezer (passa do limite de um int)
    StructField("genero", StringType()),
    StructField("artista", StringType()),
    StructField("duracao_ouvida_s", IntegerType()), # segundos ouvidos, até a duração da faixa
    StructField("instante", TimestampType()),
])
FORMATO_INSTANTE = "%Y-%m-%d %H:%M:%S"   # o mesmo de timestampFormat na leitura
```

In [34]:
```
# Gerador de eventos
SEMENTE = 42
INICIO_SIMULACAO = datetime(2026, 10, 21, 10, 0)

sorteio = np.random.default_rng(SEMENTE)
proximo_id_evento = 1


def gerar_eventos(n, inicio):
    """Sorteia n reproduções das faixas de faixas_fonte, com instantes crescentes a partir de inicio.

    A chance de cada faixa segue o rank da Deezer (faixa mais popular, mais tocada). Entre um evento e o
    seguinte passam de 5 a 30 segundos; a duração ouvida vai de 30 segundos até a faixa inteira.
    """
    global proximo_id_evento
    pesos = faixas_fonte["rank"] / faixas_fonte["rank"].sum()
    escolhidas = faixas_fonte.iloc[sorteio.choice(len(faixas_fonte), size=n, p=pesos)]
    intervalos = np.cumsum(sorteio.integers(5, 31, size=n))
    lote = []
    for (_, faixa), segundos in zip(escolhidas.iterrows(), intervalos):
        duracao = int(faixa["duracao_s"])
        lote.append({
            "id_evento": proximo_id_evento,
            "id_faixa": int(faixa["id_faixa"]),
            "genero": faixa["genero"],
            "artista": faixa["artista"],
            "duracao_ouvida_s": int(sorteio.integers(min(30, duracao), duracao + 1)),
            "instante": (inicio +
```

```
timedelta(seconds=int(segundos))).strftime(FORMATO_INSTANTE),
        })
        proximo_id_evento += 1
    return lote
```

In [35]:
```
# Publicação de um lote
lotes_publicados = 0


def publicar_lote(eventos):
    """Grava o lote em JSON (uma linha por evento) em PASTA_PUBLICACAO e
move para PASTA_ENTRADA.

    O arquivo só aparece na entrada depois de escrito e fechado, então o
Spark nunca lê um lote pela metade.
    """
    global lotes_publicados
    lotes_publicados += 1
    nome = f"lote_{lotes_publicados:03d}.json"
    arquivo = PASTA_PUBLICACAO / nome
    with arquivo.open("w", encoding="utf-8") as saida:
        for evento in eventos:
            saida.write(json.dumps(evento, ensure_ascii=False) + "\n")
    arquivo.replace(PASTA_ENTRADA / nome)
    print(f"Lote {lotes_publicados} publicado: {nome}, {len(eventos)}
eventos, "
          f"de {eventos[0]['instante']} a {eventos[-1]['instante']}")
    return PASTA_ENTRADA / nome
```

In [36]:
```
# Leitura em streaming
eventos = (
    spark.readStream
    .schema(esquema_eventos)
    .option("timestampFormat", "yyyy-MM-dd HH:mm:ss")
    .json(str(PASTA_ENTRADA))
)
print("É um DataFrame em streaming?", eventos.isStreaming)
eventos.printSchema()
```

```
É um DataFrame em streaming? True
root
 |-- id_evento: integer (nullable = true)
 |-- id_faixa: long (nullable = true)
 |-- genero: string (nullable = true)
 |-- artista: string (nullable = true)
 |-- duracao_ouvida_s: integer (nullable = true)
 |-- instante: timestamp (nullable = true)
```

In [37]:
```
# Consulta de recebimento
TABELA_RECEBIMENTO = "recebimento_" + uuid4().hex
recebimento = (
    eventos.agg(
        F.count("*").alias("eventos_recebidos"),
        F.min("instante").alias("primeiro_instante"),
        F.max("instante").alias("ultimo_instante"),
```

```
    )
    .writeStream.format("memory")
    .queryName(TABELA_RECEBIMENTO)
    .outputMode("complete")
    .option("checkpointLocation", str(PASTA_CHECKPOINT / "recebimento"))
    .start()
)
print("Consulta ativa?", recebimento.isActive)
print("Arquivos na entrada:", sorted(p.name for p in
PASTA_ENTRADA.iterdir()))
spark.table(TABELA_RECEBIMENTO).show(truncate=False)
```

```
Consulta ativa? True
Arquivos na entrada: []
+-----------------+-----------------+---------------+
|eventos_recebidos|primeiro_instante|ultimo_instante|
+-----------------+-----------------+---------------+
+-----------------+-----------------+---------------+
```

In [38]:

```
# 1º lote
primeiro_lote = gerar_eventos(30, INICIO_SIMULACAO)
publicar_lote(primeiro_lote)
display(pd.DataFrame(primeiro_lote).head(10))
```

```
Lote 1 publicado: lote_001.json, 30 eventos, de 2026-10-21 10:00:28 a 2026-1
0-21 10:09:05
```

	id_evento	id_faixa	genero	artista	duracao_ouvida_s	instante
0	1	4250206701	Asian Music	JISOO	86	2026-10-21 10:00:28
1	2	1457033832	Folk	Aoife O'Donovan	177	2026-10-21 10:00:52
2	3	96774994	Brazilian Music	Rico Dalasam	167	2026-10-21 10:01:06
3	4	722093222	Metal	TOOL	139	2026-10-21 10:01:36
4	5	73707710	Rap/Hip Hop	DJ Snake	195	2026-10-21 10:01:51
5	6	2638201552	Latin Music	FloyyMenor	85	2026-10-21 10:02:04
6	7	4275874622	African Music	FOLA	68	2026-10-21 10:02:32
7	8	3614444782	Asian Music	LE SSERAFIM	127	2026-10-21 10:02:46
8	9	2350185445	Reggaeton	Quevedo	129	2026-10-21 10:02:52
9	10	3776492922	Reggae	Nightmares On Wax	61	2026-10-21 10:03:09

In [39]:
```
# Recebimento do 1º lote
recebimento.processAllAvailable()
spark.table(TABELA_RECEBIMENTO).show(truncate=False)
print("Eventos por gênero no 1º lote:")
display(pd.DataFrame(primeiro_lote)
["genero"].value_counts().rename("eventos").to_frame())
```

```
+-----------------+-------------------+-------------------+
|eventos_recebidos|primeiro_instante  |ultimo_instante    |
+-----------------+-------------------+-------------------+
|30               |2026-10-21 10:00:28|2026-10-21 10:09:05|
+-----------------+-------------------+-------------------+

Eventos por gênero no 1º lote:
```

genero	eventos
Asian Music	3
Folk	2
Metal	2
Reggae	2
Latin Music	2
African Music	2
Reggaeton	2
Blues	2
Pop	2
Films/Games	2
Brazilian Music	1
Kids	1
Christian	1
Rap/Hip Hop	1
Dance	1
Salsa	1
Alternative	1
Cumbia	1
Rock	1

In [40]:
```
# Encerramento da consulta de recebimento
recebimento.stop()
spark.catalog.dropTempView(TABELA_RECEBIMENTO)
print("Consulta de recebimento ativa?", recebimento.isActive)
```

```
Consulta de recebimento ativa? False
```

10. Aplicação de janelas temporais

- Utilize uma janela **Tumbling** (intervalos sem sobreposição) ou **Sliding** (intervalos que podem se sobrepor).
- Informe o tipo e a duração da janela e o deslocamento, quando aplicável.
- Implemente uma agregação coerente com a análise, como contagem, soma ou média.
- Explique brevemente o que será calculado em cada intervalo.

O foco é demonstrar a compreensão do conceito; não é necessário justificar tecnicamente a duração escolhida.

Resposta do grupo:

Usamos uma única janela **tumbling de 5 minutos**, sem sobreposição. Cada intervalo inclui seu início e exclui seu fim: um evento às 10:05:00 entra em 10:05–10:10, e não em 10:00–10:05. O deslocamento é igual à duração (5 minutos), implícito nesse tipo de janela.

Contamos as reproduções por `genero` usando o instante do evento (`instante`), conforme o objetivo da seção 8. A tabela mostra a contagem de cada gênero em cada intervalo e permite identificar os gêneros com maior contagem, inclusive empates. As contagens representam eventos simulados, não audiência real da Deezer. O modo `complete` reexibe toda a tabela a cada lote; a saída `memory` permite mostrar os resultados na célula e no PDF.

In [41]:
```
# Agregação por gênero em janela tumbling de cinco minutos
from uuid import uuid4
from pyspark.sql import functions as F

if not eventos.isStreaming:
    raise ValueError("Execute a seção 9: eventos deve ser um DataFrame em streaming.")
# Interrompe consultas remanescentes antes de iniciar a consulta desta seção.
for consulta_anterior in spark.streams.active:
    consulta_anterior.stop()

agregado_janelas = eventos.groupBy(
    F.window("instante", "5 minutes"), "genero"
).agg(F.count("*").alias("reproducoes"))
```

In [42]:
```
# Consulta em memória com estado das janelas
# Nome único para poder rodar a seção novamente.
# O checkpoint é a pasta onde o Spark guarda o progresso desta consulta.
TABELA_JANELAS = "janelas_" + uuid4().hex
consulta = (
    agregado_janelas.writeStream
    .outputMode("complete")
    .format("memory")
    .queryName(TABELA_JANELAS)
    .option("checkpointLocation", str(PASTA_CHECKPOINT / TABELA_JANELAS))
    .start()
)
print(f"Consulta ativa: {consulta.isActive}; tabela em memória: {TABELA_JANELAS}")
```

```
Consulta ativa: True; tabela em memória: janelas_8722e9edd6f7446598a1dbcb3b99c443
```

In [43]:
```
# Tabela das janelas após o primeiro lote
consulta.processAllAvailable()
tabela_primeiro_lote = spark.table(TABELA_JANELAS)
quantidade_grupos = tabela_primeiro_lote.count()
# O limite é o número real de grupos, para que show() não esconda linhas
além das primeiras 20.
tabela_primeiro_lote.select(
    F.col("window.start").alias("inicio"),
    F.col("window.end").alias("fim"), "genero", "reproducoes"
).orderBy("inicio", F.desc("reproducoes"), "genero").show(n=max(1,
quantidade_grupos), truncate=False)

# Somamos as contagens da tabela para conferir quantos eventos foram
recebidos.
total_reproducoes = tabela_primeiro_lote.agg(
    F.sum("reproducoes").alias("total")
).first()["total"]
print(f"Total de reproduções recebidas: {total_reproducoes or 0}")
# A consulta permanece ativa para os novos lotes e o encerramento da seção
11.
```

```
+-------------------+-------------------+---------------+-----------+
|inicio             |fim                |genero         |reproducoes|
+-------------------+-------------------+---------------+-----------+
|2026-10-21 10:00:00|2026-10-21 10:05:00|Asian Music    |2          |
|2026-10-21 10:00:00|2026-10-21 10:05:00|Folk           |2          |
|2026-10-21 10:00:00|2026-10-21 10:05:00|African Music  |1          |
|2026-10-21 10:00:00|2026-10-21 10:05:00|Blues          |1          |
|2026-10-21 10:00:00|2026-10-21 10:05:00|Brazilian Music|1          |
|2026-10-21 10:00:00|2026-10-21 10:05:00|Christian      |1          |
|2026-10-21 10:00:00|2026-10-21 10:05:00|Dance          |1          |
|2026-10-21 10:00:00|2026-10-21 10:05:00|Films/Games    |1          |
|2026-10-21 10:00:00|2026-10-21 10:05:00|Kids           |1          |
|2026-10-21 10:00:00|2026-10-21 10:05:00|Latin Music    |1          |
|2026-10-21 10:00:00|2026-10-21 10:05:00|Metal          |1          |
|2026-10-21 10:00:00|2026-10-21 10:05:00|Rap/Hip Hop    |1          |
|2026-10-21 10:00:00|2026-10-21 10:05:00|Reggae         |1          |
|2026-10-21 10:00:00|2026-10-21 10:05:00|Reggaeton      |1          |
|2026-10-21 10:05:00|2026-10-21 10:10:00|Pop            |2          |
|2026-10-21 10:05:00|2026-10-21 10:10:00|African Music  |1          |
|2026-10-21 10:05:00|2026-10-21 10:10:00|Alternative    |1          |
|2026-10-21 10:05:00|2026-10-21 10:10:00|Asian Music    |1          |
|2026-10-21 10:05:00|2026-10-21 10:10:00|Blues          |1          |
|2026-10-21 10:05:00|2026-10-21 10:10:00|Cumbia         |1          |
|2026-10-21 10:05:00|2026-10-21 10:10:00|Films/Games    |1          |
|2026-10-21 10:05:00|2026-10-21 10:10:00|Latin Music    |1          |
|2026-10-21 10:05:00|2026-10-21 10:10:00|Metal          |1          |
|2026-10-21 10:05:00|2026-10-21 10:10:00|Reggae         |1          |
|2026-10-21 10:05:00|2026-10-21 10:10:00|Reggaeton      |1          |
|2026-10-21 10:05:00|2026-10-21 10:10:00|Rock           |1          |
|2026-10-21 10:05:00|2026-10-21 10:10:00|Salsa          |1          |
+-------------------+-------------------+---------------+-----------+

Total de reproduções recebidas: 30
```

11. Resultados e interpretação

Apresente os resultados do processamento e demonstre como a análise se comporta com a chegada de novos registros.

In [44]:
```
# Preparação da comparação entre lotes
from datetime import datetime
from pyspark.sql import functions as F

if not consulta.isActive:
    raise RuntimeError("Execute as seções 9 e 10 antes de demonstrar os
novos lotes.")


def ler_resultados_janelas():
    """Espera os arquivos disponíveis e copia o resultado atual para
comparação."""
    consulta.processAllAvailable()
    return (
        spark.table(TABELA_JANELAS)
        .select(
            F.col("window.start").alias("inicio"),
            F.col("window.end").alias("fim"),
            "genero", "reproducoes",
        )
        .orderBy("inicio", F.desc("reproducoes"), "genero")
        .toPandas()
    )


resultados_atuais = ler_resultados_janelas()
lotes_demonstrados = [primeiro_lote]


def publicar_proximo_lote(quantidade=30):
    """Publica novas reproduções e mostra o resultado completo e as
mudanças por janela."""
    global resultados_atuais
    if not consulta.isActive:
        raise RuntimeError("A consulta foi encerrada. Execute novamente as
seções 9 e 10.")
    if not isinstance(quantidade, int) or quantidade <= 0:
        raise ValueError("A quantidade de eventos deve ser um inteiro
positivo.")

    anteriores = resultados_atuais
    inicio = datetime.strptime(lotes_demonstrados[-1][-1]["instante"],
FORMATO_INSTANTE)
    novo_lote = gerar_eventos(quantidade, inicio)
    publicar_lote(novo_lote)
    atuais = ler_resultados_janelas()

    chaves = ["inicio", "fim", "genero"]
```

```
    mudancas = atuais.merge(
        anteriores, on=chaves, how="outer", suffixes=("_depois", "_antes")
    ).fillna({"reproducoes_antes": 0, "reproducoes_depois": 0})
    for coluna in ("reproducoes_antes", "reproducoes_depois"):
        mudancas[coluna] = mudancas[coluna].astype("int64")
    mudancas["novas_reproducoes"] = mudancas["reproducoes_depois"] -
mudancas["reproducoes_antes"]
    assert mudancas["novas_reproducoes"].ge(0).all(), "Uma contagem
anterior diminuiu."
    assert int(mudancas["novas_reproducoes"].sum()) == len(novo_lote), "O
lote não foi contado integralmente."

    lotes_demonstrados.append(novo_lote)
    total_esperado = sum(len(lote) for lote in lotes_demonstrados)
    total_recebido = int(atuais["reproducoes"].sum())
    assert total_recebido == total_esperado, "Total recebido diferente dos
lotes publicados."

    resumo = mudancas.groupby(["inicio", "fim"], as_index=False)[
        ["reproducoes_antes", "novas_reproducoes", "reproducoes_depois"]
    ].sum()
    print(f"Após o lote {len(lotes_demonstrados)}: {total_recebido}
reproduções recebidas.")
    with pd.option_context("display.max_rows", None):
        print("Contagens por gênero e janela:")
        display(atuais)
        print("Mudanças por janela (início incluído, fim excluído):")
        display(resumo)
    resultados_atuais = atuais
    return novo_lote
```

In [45]:
```
# Segundo lote: atualização de uma janela existente e abertura de novos
intervalos
segundo_lote = publicar_proximo_lote()
resultados_apos_segundo_lote = resultados_atuais.copy()
```

```
Lote 2 publicado: lote_002.json, 30 eventos, de 2026-10-21 10:09:34 a 2026-1
0-21 10:17:57
Após o lote 2: 60 reproduções recebidas.
Contagens por gênero e janela:
```

	inicio	fim	genero	reproducoes
0	2026-10-21 10:00:00	2026-10-21 10:05:00	Asian Music	2
1	2026-10-21 10:00:00	2026-10-21 10:05:00	Folk	2
2	2026-10-21 10:00:00	2026-10-21 10:05:00	African Music	1
3	2026-10-21 10:00:00	2026-10-21 10:05:00	Blues	1
4	2026-10-21 10:00:00	2026-10-21 10:05:00	Brazilian Music	1
5	2026-10-21 10:00:00	2026-10-21 10:05:00	Christian	1
6	2026-10-21 10:00:00	2026-10-21 10:05:00	Dance	1
7	2026-10-21 10:00:00	2026-10-21 10:05:00	Films/Games	1
8	2026-10-21 10:00:00	2026-10-21 10:05:00	Kids	1
9	2026-10-21 10:00:00	2026-10-21 10:05:00	Latin Music	1
10	2026-10-21 10:00:00	2026-10-21 10:05:00	Metal	1
11	2026-10-21 10:00:00	2026-10-21 10:05:00	Rap/Hip Hop	1
12	2026-10-21 10:00:00	2026-10-21 10:05:00	Reggae	1
13	2026-10-21 10:00:00	2026-10-21 10:05:00	Reggaeton	1
14	2026-10-21 10:05:00	2026-10-21 10:10:00	Metal	2
15	2026-10-21 10:05:00	2026-10-21 10:10:00	Pop	2
16	2026-10-21 10:05:00	2026-10-21 10:10:00	Reggae	2
17	2026-10-21 10:05:00	2026-10-21 10:10:00	African Music	1
18	2026-10-21 10:05:00	2026-10-21 10:10:00	Alternative	1
19	2026-10-21 10:05:00	2026-10-21 10:10:00	Asian Music	1
20	2026-10-21 10:05:00	2026-10-21 10:10:00	Blues	1
21	2026-10-21 10:05:00	2026-10-21 10:10:00	Cumbia	1
22	2026-10-21 10:05:00	2026-10-21 10:10:00	Films/Games	1
23	2026-10-21 10:05:00	2026-10-21 10:10:00	Latin Music	1
24	2026-10-21 10:05:00	2026-10-21 10:10:00	Reggaeton	1
25	2026-10-21 10:05:00	2026-10-21 10:10:00	Rock	1
26	2026-10-21 10:05:00	2026-10-21 10:10:00	Salsa	1
27	2026-10-21 10:05:00	2026-10-21 10:10:00	Traditional Mexicano	1
28	2026-10-21 10:10:00	2026-10-21 10:15:00	Dance	2
29	2026-10-21 10:10:00	2026-10-21 10:15:00	Metal	2
30	2026-10-21 10:10:00	2026-10-21 10:15:00	Pop	2
31	2026-10-21 10:10:00	2026-10-21 10:15:00	R&B	2
32	2026-10-21 10:10:00	2026-10-21 10:15:00	Salsa	2

	inicio	fim	genero	reproducoes
33	2026-10-21 10:10:00	2026-10-21 10:15:00	African Music	1
34	2026-10-21 10:10:00	2026-10-21 10:15:00	Alternative	1
35	2026-10-21 10:10:00	2026-10-21 10:15:00	Asian Music	1
36	2026-10-21 10:10:00	2026-10-21 10:15:00	Brazilian Music	1
37	2026-10-21 10:10:00	2026-10-21 10:15:00	Electro	1
38	2026-10-21 10:10:00	2026-10-21 10:15:00	Films/Games	1
39	2026-10-21 10:10:00	2026-10-21 10:15:00	Folk	1
40	2026-10-21 10:10:00	2026-10-21 10:15:00	Traditional Mexicano	1
41	2026-10-21 10:15:00	2026-10-21 10:20:00	Rock	2
42	2026-10-21 10:15:00	2026-10-21 10:20:00	Asian Music	1
43	2026-10-21 10:15:00	2026-10-21 10:20:00	Country	1
44	2026-10-21 10:15:00	2026-10-21 10:20:00	Electro	1
45	2026-10-21 10:15:00	2026-10-21 10:20:00	Pop	1
46	2026-10-21 10:15:00	2026-10-21 10:20:00	Rap/Hip Hop	1
47	2026-10-21 10:15:00	2026-10-21 10:20:00	Reggae	1
48	2026-10-21 10:15:00	2026-10-21 10:20:00	Soul & Funk	1

Mudanças por janela (início incluído, fim excluído):

	inicio	fim	reproducoes_antes	novas_reproducoes	reproducoes_depo
0	2026-10-21 10:00:00	2026-10-21 10:05:00	16	0	1
1	2026-10-21 10:05:00	2026-10-21 10:10:00	14	3	1
2	2026-10-21 10:10:00	2026-10-21 10:15:00	0	18	1
3	2026-10-21 10:15:00	2026-10-21 10:20:00	0	9	

In [46]:
```
# Terceiro lote: novas reproduções e comparação das contagens acumuladas
terceiro_lote = publicar_proximo_lote()
resultados_apos_terceiro_lote = resultados_atuais.copy()
```

```
Lote 3 publicado: lote_003.json, 30 eventos, de 2026-10-21 10:18:11 a 2026-1
0-21 10:26:15
Após o lote 3: 90 reproduções recebidas.
Contagens por gênero e janela:
```

	inicio	fim	genero	reproducoes
0	2026-10-21 10:00:00	2026-10-21 10:05:00	Asian Music	2
1	2026-10-21 10:00:00	2026-10-21 10:05:00	Folk	2
2	2026-10-21 10:00:00	2026-10-21 10:05:00	African Music	1
3	2026-10-21 10:00:00	2026-10-21 10:05:00	Blues	1
4	2026-10-21 10:00:00	2026-10-21 10:05:00	Brazilian Music	1
5	2026-10-21 10:00:00	2026-10-21 10:05:00	Christian	1
6	2026-10-21 10:00:00	2026-10-21 10:05:00	Dance	1
7	2026-10-21 10:00:00	2026-10-21 10:05:00	Films/Games	1
8	2026-10-21 10:00:00	2026-10-21 10:05:00	Kids	1
9	2026-10-21 10:00:00	2026-10-21 10:05:00	Latin Music	1
10	2026-10-21 10:00:00	2026-10-21 10:05:00	Metal	1
11	2026-10-21 10:00:00	2026-10-21 10:05:00	Rap/Hip Hop	1
12	2026-10-21 10:00:00	2026-10-21 10:05:00	Reggae	1
13	2026-10-21 10:00:00	2026-10-21 10:05:00	Reggaeton	1
14	2026-10-21 10:05:00	2026-10-21 10:10:00	Metal	2
15	2026-10-21 10:05:00	2026-10-21 10:10:00	Pop	2
16	2026-10-21 10:05:00	2026-10-21 10:10:00	Reggae	2
17	2026-10-21 10:05:00	2026-10-21 10:10:00	African Music	1
18	2026-10-21 10:05:00	2026-10-21 10:10:00	Alternative	1
19	2026-10-21 10:05:00	2026-10-21 10:10:00	Asian Music	1
20	2026-10-21 10:05:00	2026-10-21 10:10:00	Blues	1
21	2026-10-21 10:05:00	2026-10-21 10:10:00	Cumbia	1
22	2026-10-21 10:05:00	2026-10-21 10:10:00	Films/Games	1
23	2026-10-21 10:05:00	2026-10-21 10:10:00	Latin Music	1
24	2026-10-21 10:05:00	2026-10-21 10:10:00	Reggaeton	1
25	2026-10-21 10:05:00	2026-10-21 10:10:00	Rock	1
26	2026-10-21 10:05:00	2026-10-21 10:10:00	Salsa	1
27	2026-10-21 10:05:00	2026-10-21 10:10:00	Traditional Mexicano	1
28	2026-10-21 10:10:00	2026-10-21 10:15:00	Dance	2
29	2026-10-21 10:10:00	2026-10-21 10:15:00	Metal	2
30	2026-10-21 10:10:00	2026-10-21 10:15:00	Pop	2
31	2026-10-21 10:10:00	2026-10-21 10:15:00	R&B	2
32	2026-10-21 10:10:00	2026-10-21 10:15:00	Salsa	2

	inicio	fim	genero	reproducoes
33	2026-10-21 10:10:00	2026-10-21 10:15:00	African Music	1
34	2026-10-21 10:10:00	2026-10-21 10:15:00	Alternative	1
35	2026-10-21 10:10:00	2026-10-21 10:15:00	Asian Music	1
36	2026-10-21 10:10:00	2026-10-21 10:15:00	Brazilian Music	1
37	2026-10-21 10:10:00	2026-10-21 10:15:00	Electro	1
38	2026-10-21 10:10:00	2026-10-21 10:15:00	Films/Games	1
39	2026-10-21 10:10:00	2026-10-21 10:15:00	Folk	1
40	2026-10-21 10:10:00	2026-10-21 10:15:00	Traditional Mexicano	1
41	2026-10-21 10:15:00	2026-10-21 10:20:00	Country	2
42	2026-10-21 10:15:00	2026-10-21 10:20:00	Electro	2
43	2026-10-21 10:15:00	2026-10-21 10:20:00	Pop	2
44	2026-10-21 10:15:00	2026-10-21 10:20:00	Rap/Hip Hop	2
45	2026-10-21 10:15:00	2026-10-21 10:20:00	Rock	2
46	2026-10-21 10:15:00	2026-10-21 10:20:00	Alternative	1
47	2026-10-21 10:15:00	2026-10-21 10:20:00	Asian Music	1
48	2026-10-21 10:15:00	2026-10-21 10:20:00	Films/Games	1
49	2026-10-21 10:15:00	2026-10-21 10:20:00	Reggae	1
50	2026-10-21 10:15:00	2026-10-21 10:20:00	Soul & Funk	1
51	2026-10-21 10:15:00	2026-10-21 10:20:00	Traditional Mexicano	1
52	2026-10-21 10:20:00	2026-10-21 10:25:00	Traditional Mexicano	4
53	2026-10-21 10:20:00	2026-10-21 10:25:00	Country	3
54	2026-10-21 10:20:00	2026-10-21 10:25:00	Kids	2
55	2026-10-21 10:20:00	2026-10-21 10:25:00	Rap/Hip Hop	2
56	2026-10-21 10:20:00	2026-10-21 10:25:00	Alternative	1
57	2026-10-21 10:20:00	2026-10-21 10:25:00	Asian Music	1
58	2026-10-21 10:20:00	2026-10-21 10:25:00	Latin Music	1
59	2026-10-21 10:20:00	2026-10-21 10:25:00	Pop	1
60	2026-10-21 10:20:00	2026-10-21 10:25:00	Reggaeton	1
61	2026-10-21 10:20:00	2026-10-21 10:25:00	Rock	1
62	2026-10-21 10:25:00	2026-10-21 10:30:00	Alternative	1
63	2026-10-21 10:25:00	2026-10-21 10:30:00	Country	1
64	2026-10-21 10:25:00	2026-10-21 10:30:00	Folk	1
65	2026-10-21 10:25:00	2026-10-21 10:30:00	Jazz	1

	inicio	fim	genero	reproducoes
66	2026-10-21 10:25:00	2026-10-21 10:30:00	Pop	1
67	2026-10-21 10:25:00	2026-10-21 10:30:00	R&B	1

Mudanças por janela (início incluído, fim excluído):

	inicio	fim	reproducoes_antes	novas_reproducoes	reproducoes_depo
0	2026-10-21 10:00:00	2026-10-21 10:05:00	16	0	1
1	2026-10-21 10:05:00	2026-10-21 10:10:00	17	0	1
2	2026-10-21 10:10:00	2026-10-21 10:15:00	18	0	1
3	2026-10-21 10:15:00	2026-10-21 10:20:00	9	7	1
4	2026-10-21 10:20:00	2026-10-21 10:25:00	0	17	1
5	2026-10-21 10:25:00	2026-10-21 10:30:00	0	6	

Resposta do grupo:

As células “Segundo lote: atualização de uma janela existente e abertura de novos intervalos” e “Terceiro lote: novas reproduções e comparação das contagens acumuladas” publicam mais 30 eventos cada, em momentos distintos, e mostram a tabela após o processamento. Com os 30 eventos do primeiro lote, o total recebido passa de 30 para 60 e depois para 90.

O segundo lote acrescenta 3 reproduções em 10:05–10:10 (de 14 para 17), 18 em 10:10–10:15 e 9 em 10:15–10:20. O terceiro acrescenta 7 em 10:15–10:20 (de 9 para 16), 17 em 10:20–10:25 e 6 em 10:25–10:30. A janela 10:00–10:05 mantém suas 16 reproduções, pois nenhum evento dos dois novos lotes pertence a esse intervalo. Essas contagens são apresentadas nos resumos “Mudanças por janela” das duas células.

As janelas são atribuídas pelo instante de cada evento, não pelo horário em que o arquivo foi publicado. Assim, uma janela já exibida recebe novas contagens quando chegam eventos cujo instante ainda pertence a ela. A tabela por gênero permite comparar quais gêneros concentram as reproduções em cada intervalo; os valores representam uma simulação com semente fixa, e não a audiência real da Deezer.

Demonstração ao vivo: antes de executar a célula de encerramento, execute `publicar_proximo_lote(10)` em uma célula de código para publicar mais dez eventos e mostrar as mudanças. A função continua a sequência de IDs e instantes do lote anterior. Para repetir a demonstração desde o começo, execute novamente as seções 9, 10 e 11.

In [47]:
```
# Encerramento da consulta e limpeza das subpastas da simulação
import shutil

consulta.stop()
spark.catalog.dropTempView(TABELA_JANELAS)
for pasta in (PASTA_ENTRADA, PASTA_PUBLICACAO, PASTA_CHECKPOINT):
    destino = pasta.resolve()
    if destino.parent != (RAIZ / "dados" / "streaming").resolve():
        raise ValueError(f"Pasta fora do diretório da simulação:
{destino}")
    if destino.exists():
        shutil.rmtree(destino)
print("Consulta ativa após o encerramento?", consulta.isActive)
print("Subpastas de entrada, publicação e checkpoint removidas; README
preservado.")
```

```
Consulta ativa após o encerramento? False
Subpastas de entrada, publicação e checkpoint removidas; README preservado.
```

12. Considerações do RA2

Responda brevemente:

- O que foi possível observar com o processamento em streaming?
- Qual foi a contribuição das janelas temporais para a análise?
- Essa abordagem faria sentido em uma aplicação real do projeto? Por quê?

Na última resposta, o grupo pode reconhecer que o streaming foi utilizado apenas como exercício didático, se esse for o caso.

Resposta do grupo:

O que observamos com o streaming. A consulta processou cada lote assim que ele chegou à pasta monitorada, sem reler os anteriores: o total recebido passou de 30 para 60 e depois para 90 reproduções, e a tabela mudou a cada lote (seção 11, células "Segundo lote: atualização de uma janela existente e abertura de novos intervalos" e "Terceiro lote: novas reproduções e comparação das contagens acumuladas"). Um lote novo pode atualizar uma janela já exibida, como 10:05–10:10, que foi de 14 para 17, e abrir janelas novas, como 10:20–10:25, com 17. A janela 10:00–10:05 ficou em 16, porque nenhum evento novo caiu nela.

Contribuição das janelas temporais. A janela tumbling de 5 minutos transformou um fluxo contínuo de eventos em intervalos comparáveis: em vez de um total único, cada intervalo mostra quantas reproduções cada gênero teve (seção 10). Como a janela usa o instante do evento, e não a hora de chegada do arquivo, um evento atrasado ainda é contado no intervalo certo. É isso que permite responder ao objetivo da seção 8: qual gênero concentra mais reproduções em cada intervalo.

Uso numa aplicação real. Faria sentido com eventos reais de reprodução, por exemplo para a Deezer ou uma gravadora perceberem em minutos um gênero ou uma faixa ganhando audiência. Neste projeto, porém, o streaming foi um exercício didático: a API pública só oferece um retrato diário dos charts, sem fluxo de reproduções, e os eventos são simulados a partir do conjunto preparado do RA1, com semente fixa. Os números mostram o funcionamento do processamento, não a audiência real.